

KX-AI Flow - Backend Architecture Documentation

Version: 1.2
Date: September 4, 2026
Technology Stack: FastAPI, SQLAlchemy, PyMongo, PostgreSQL, MongoDB, LangChain **Document**
Maintained By: Sanjay Kumar Prajapati (AVP - Software Development)

Table of Contents

- 1. [System Overview](#)
 - 2. [Architecture Layers](#)
 - 3. [Component Architecture](#)
 - 4. [Design Patterns](#)
 - 5. [Database Schema](#) 5a. [MongoDB Collections](#)
 - 6. [API Architecture](#)
 - 7. [Workflow Engine](#)
 - 8. [Authentication & Authorization](#)
 - 9. [Sequence Diagrams](#)
 - 10. [Deployment Architecture](#)
-

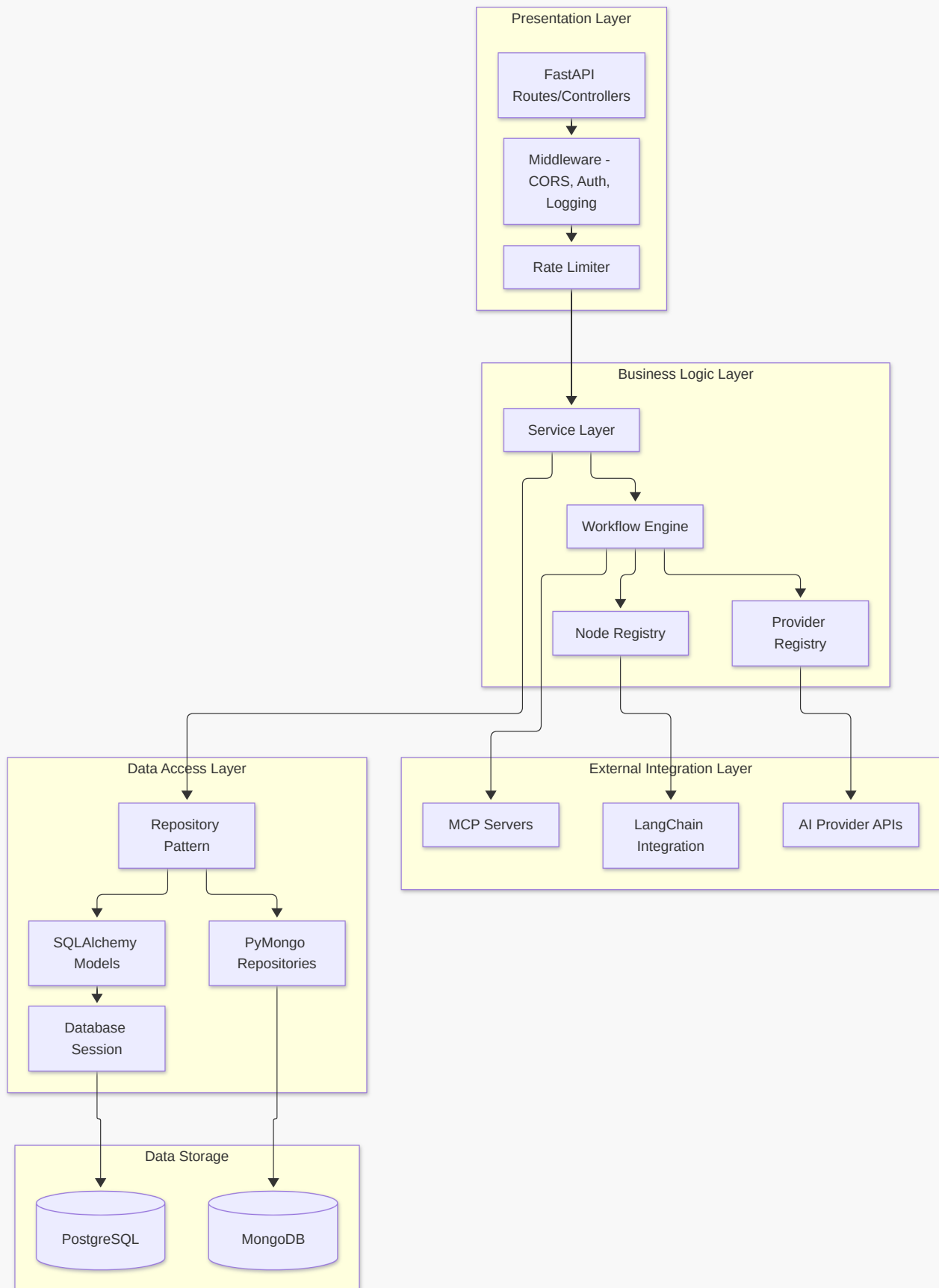
System Overview

KX-AI Flow is a workflow orchestration platform that enables users to create, manage, and execute AI-powered agent workflows. The backend is built with FastAPI and follows a clean, layered architecture with clear separation of concerns.

Key Features

- JWT-based authentication
 - Multi-tenant project management
 - Dual database backend support (PostgreSQL or MongoDB)
 - Dynamic workflow execution engine
 - Async flow-run queue (`/flows/{flow_id}/runs`) with retries/idempotency
 - AI agent orchestration with multiple providers (OpenAI, Anthropic, etc.)
 - MCP (Model Context Protocol) integration
 - Embed client token flow and flow-scoped authorization
 - WhatsApp webhook ingestion and signature verification
 - Real-time chat sessions
 - Usage tracking and logging
 - Rate limiting
-

Architecture Layers

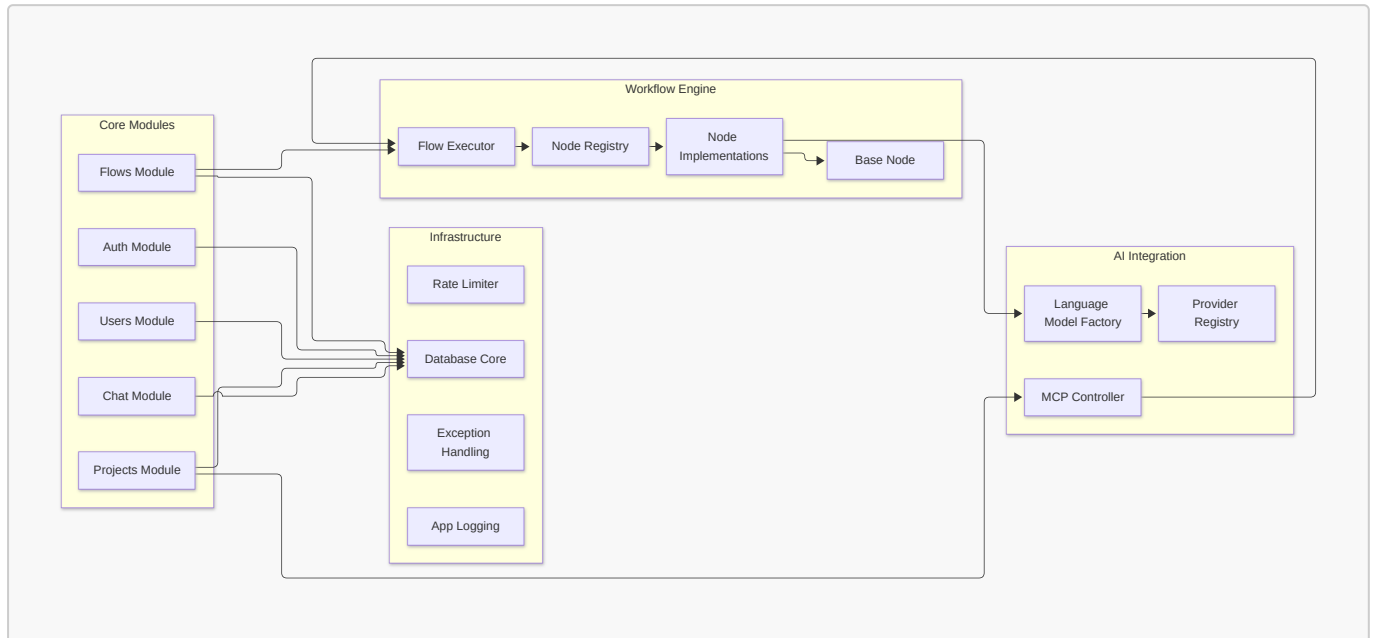


Layer Responsibilities

1. **Presentation Layer:** HTTP request handling, routing, middleware
2. **Business Logic Layer:** Core business rules, workflow orchestration

3. **Data Access Layer:** Database operations, ORM models
 4. **External Integration Layer:** Third-party API integrations
 5. **Data Storage:** Configurable backend (**DB_BACKEND**) using PostgreSQL or MongoDB
-

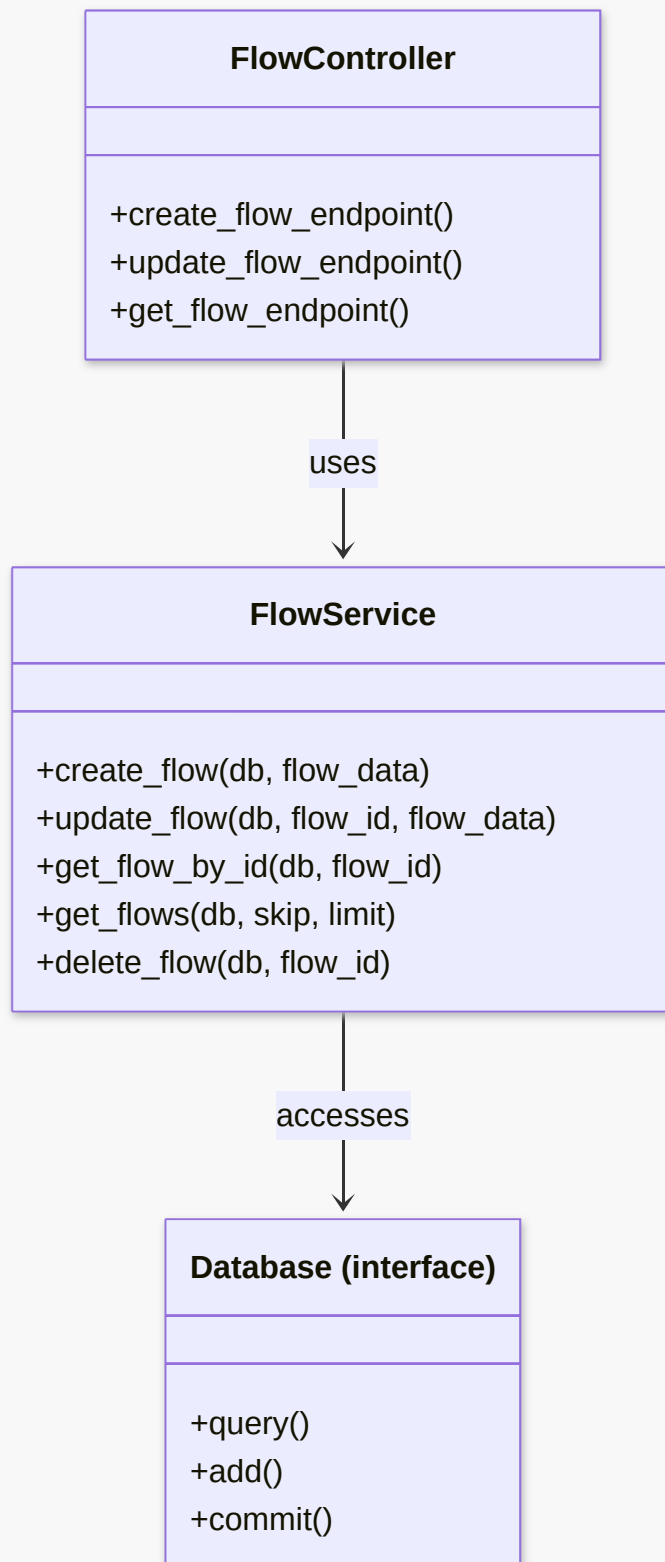
Component Architecture



Design Patterns

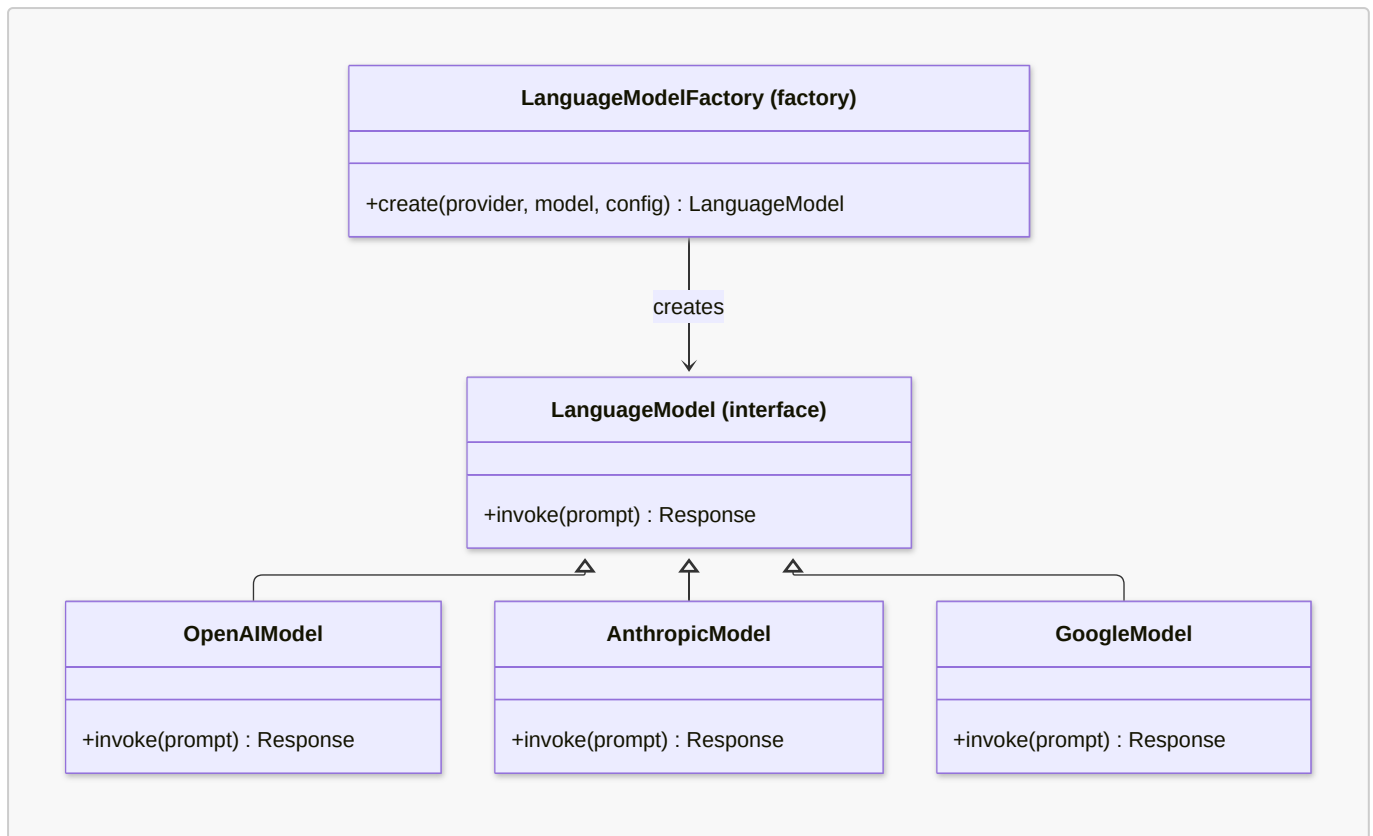
1. Repository Pattern

Location: Service Layer (**auth/service.py**, **flows/service.py**, **users/service.py**)



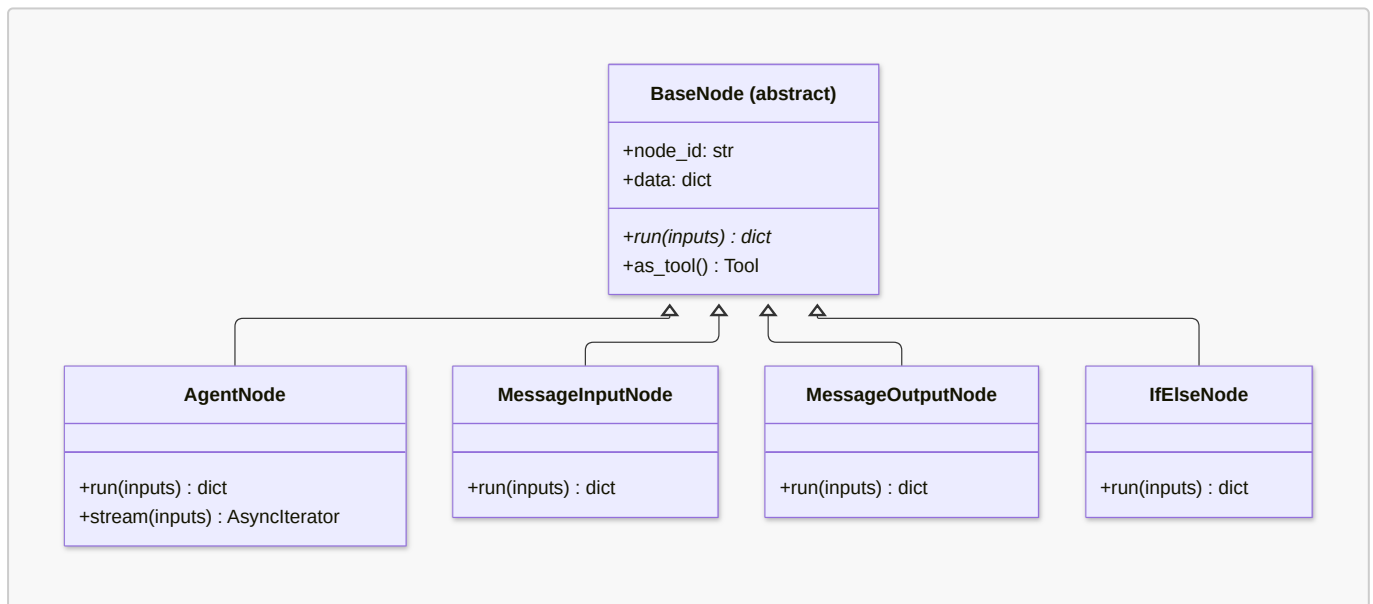
2. Factory Pattern

Location: `workflow_engine/nodes/agents/language_model.py`, `flow_executor.py`



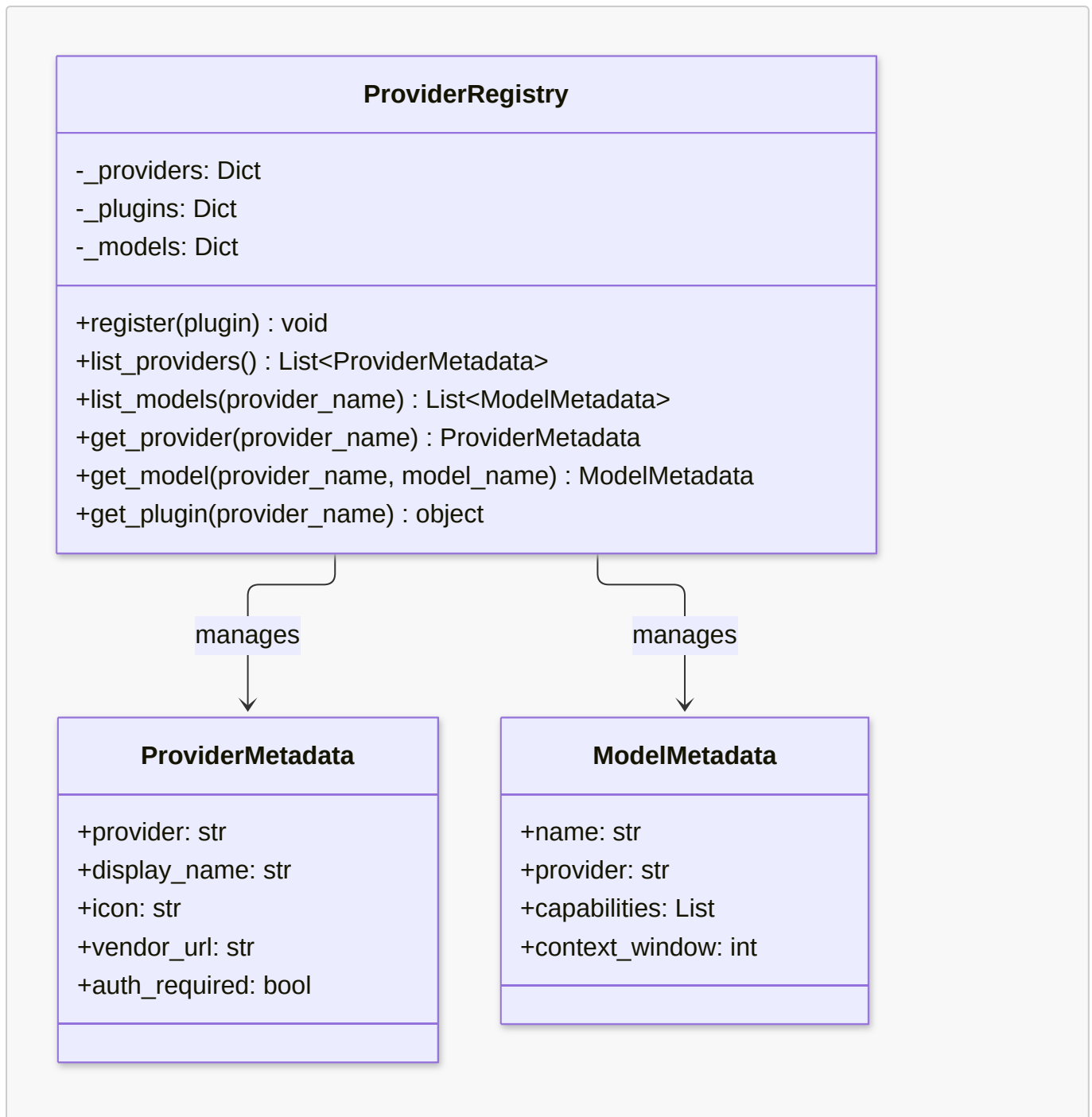
3. Strategy Pattern

Location: `workflow_engine/base_node.py` and node implementations



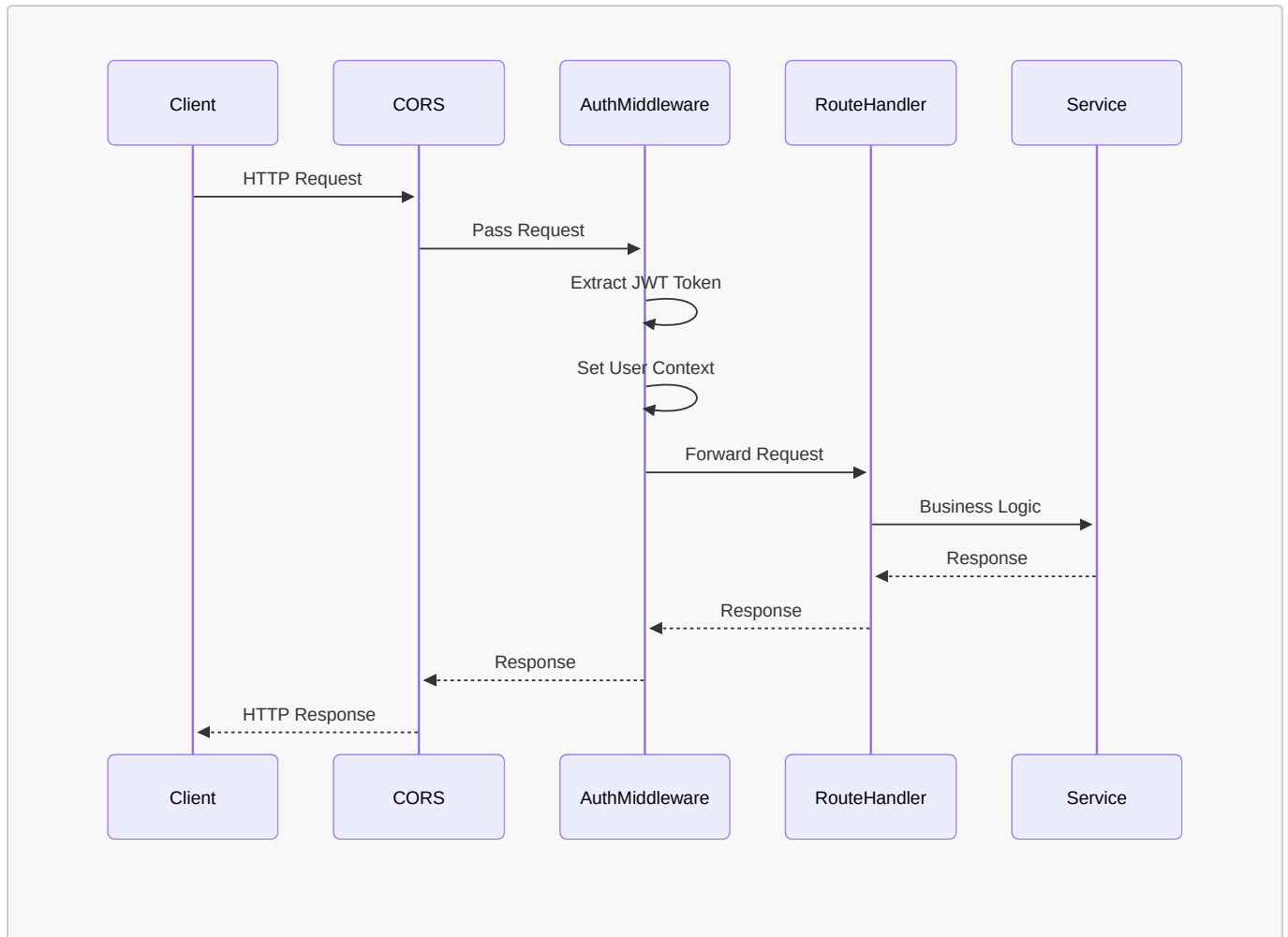
4. Registry Pattern

Location: `workflow_engine/nodes/metadata/provider_registry.py`



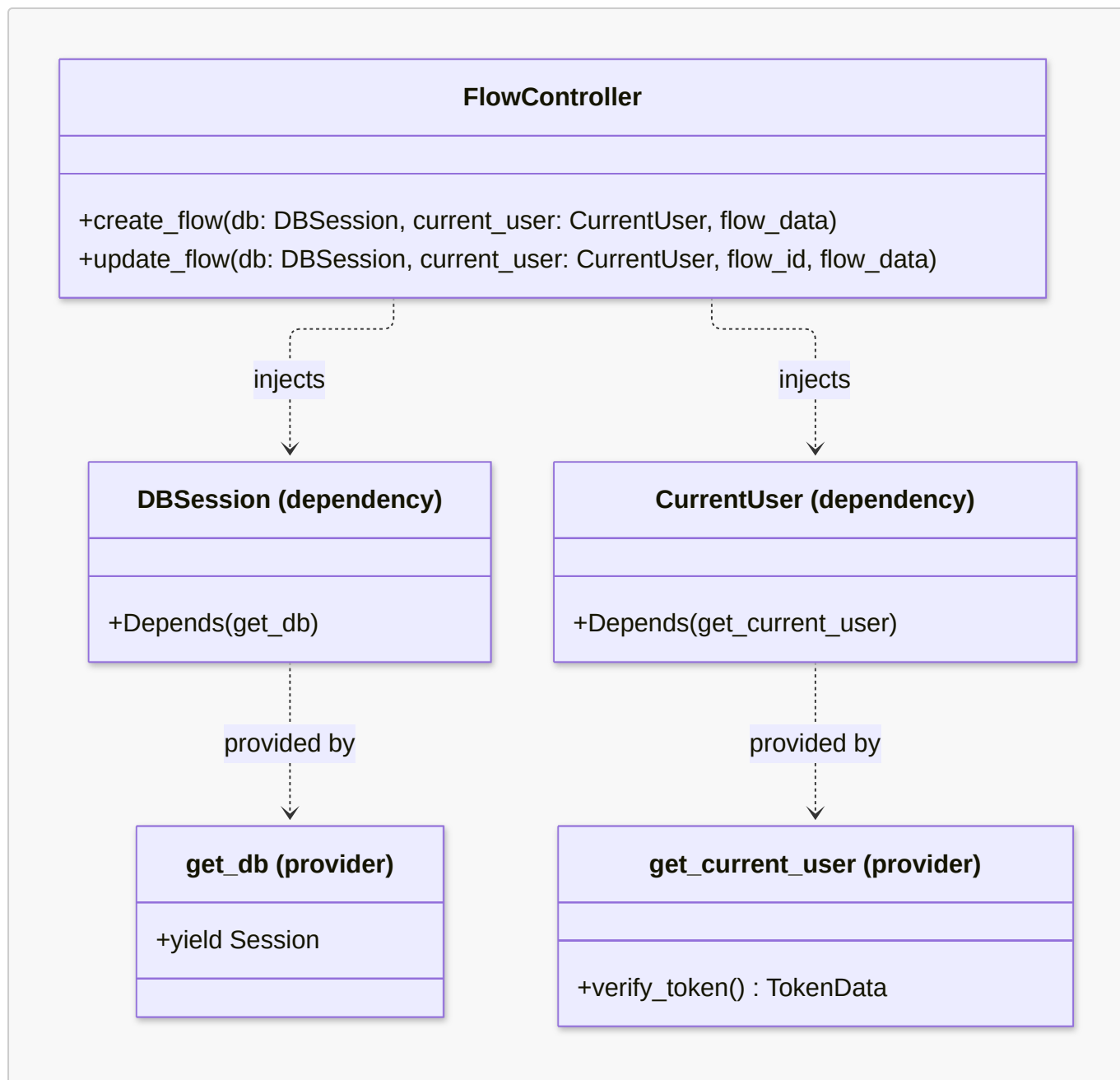
5. Middleware Pattern

Location: `main.py`



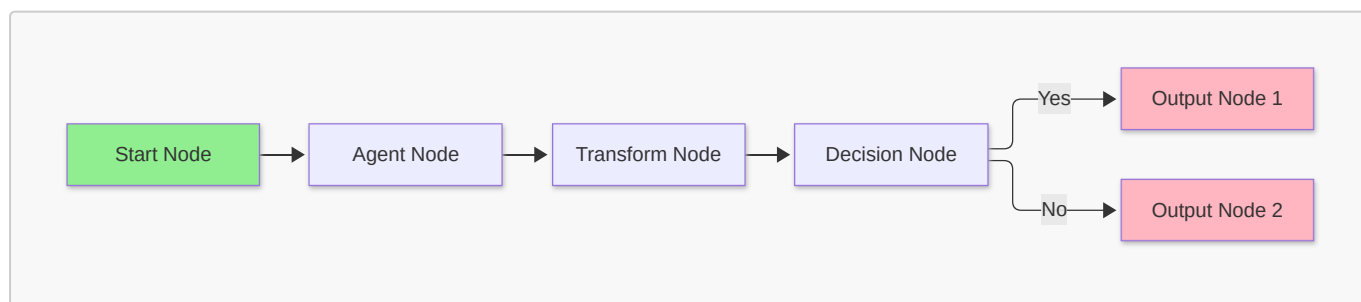
6. Dependency Injection

Location: Controllers using FastAPI's `Depends()`



7. Chain of Responsibility

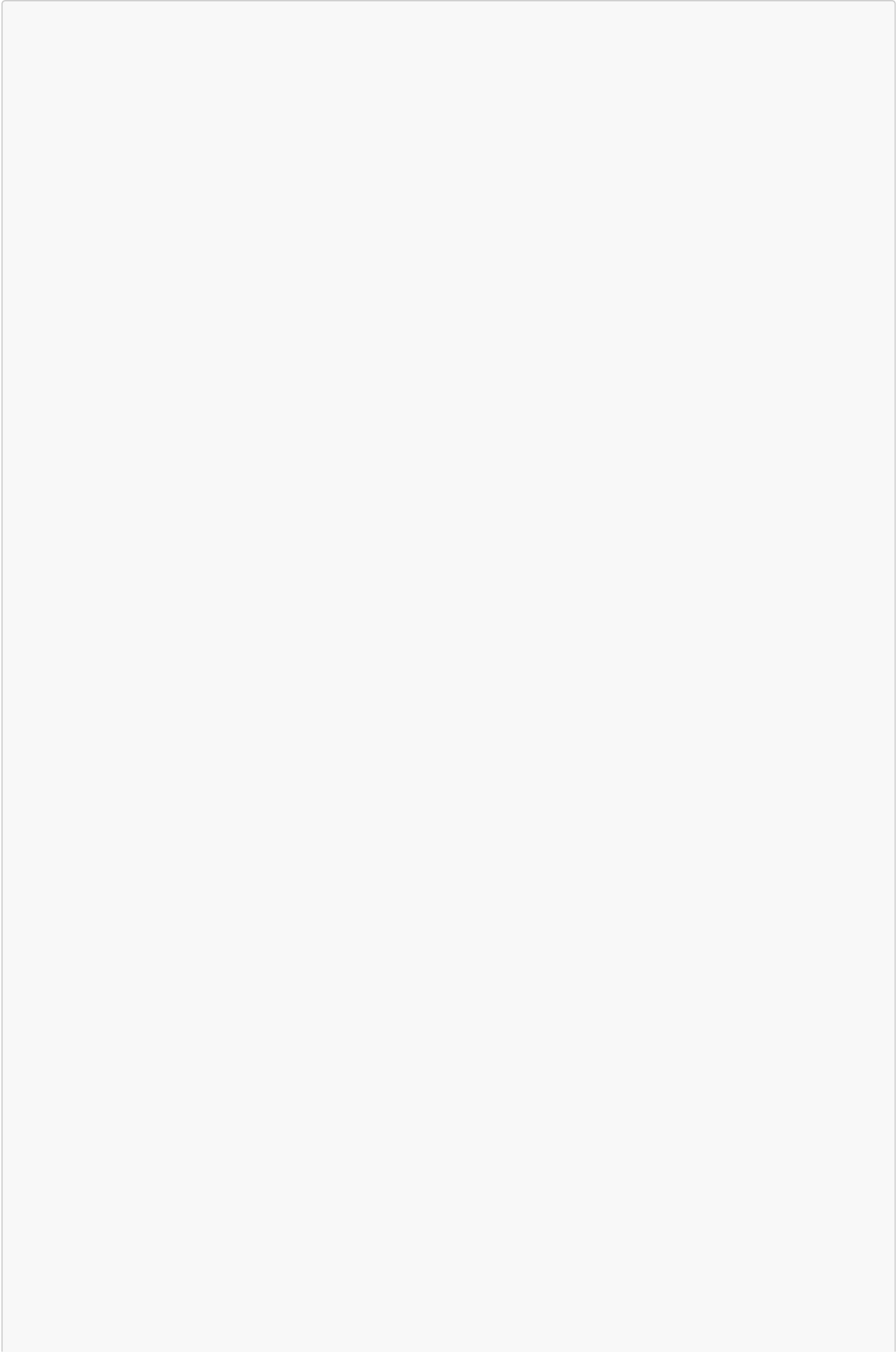
Location: `workflow_engine/flow_executor.py`

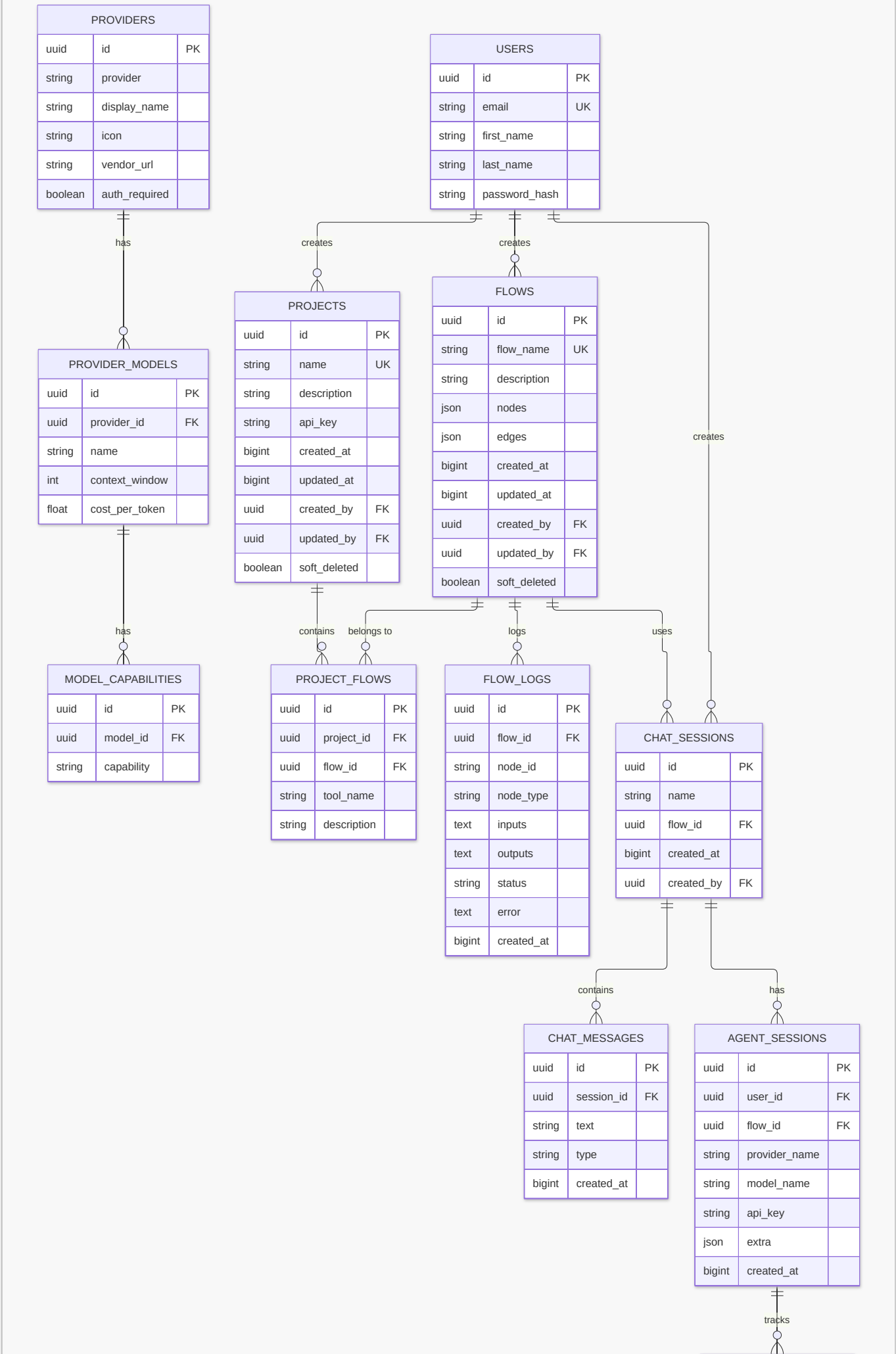


Database Schema

Note: The system uses a repository abstraction over two persistence backends. The ER diagram below represents the logical domain model; in practice this is stored in SQL tables (PostgreSQL) or Mongo

collections/documents (MongoDB).





USAGE_LOGS		
uuid	id	PK
uuid	session_id	FK
uuid	flow_id	FK
uuid	provider_id	FK
uuid	model_id	FK
string	capability	
int	tokens_used	
int	latency_ms	
float	cost	
bigint	created_at	

Database Design Principles

- 1. **Audit Trail:** All entities extend `AuditBase` with `created_at`, `updated_at`, `created_by`, `updated_by`
- 2. **Soft Deletion:** Uses `soft_deleted` flag instead of hard deletes
- 3. **UUID Primary Keys:** All tables use UUID for distributed system compatibility
- 4. **Timestamps:** BigInteger millisecond timestamps for precision
- 5. **JSON Storage:** Complex data (nodes, edges) stored as JSON for flexibility

MongoDB Collections

Source: `src/database/repositories/mongo.py` (document shape/CRUD) and `src/database/core.py` (`DatabaseContext`, `ensure_mongo_indexes()`). Selected when `DB_BACKEND=mongo` (or `mongodb`). Every collection uses a **string-ified UUID as `_id`** (no native ObjectId), so the same identifier works whether the app is pointed at PostgreSQL or MongoDB. Audit fields (`g_creation_time`, `g_modify_time`, `g_created_by_id`, `g_modified_by_id`, `g_soft_delete`) are hand-stamped in the repository layer (`_audit_doc` / `_touch_doc` helpers) rather than by a Mongo-side hook, mirroring the SQL `AuditBase` convention. `g_soft_delete` is `"Y"/"N"`; most reads filter `g_soft_delete: "N"`.

flows

Repository: `MongoFlowRepository`. One document per workflow definition (nodes/edges graph).

Field	Type	Notes
<code>_id / id</code>	string (UUID)	Primary key
<code>flow_name</code>	string	Unique among non-deleted flows
<code>description</code>	string	
<code>nodes</code>	array	Node graph (arbitrary JSON per node)
<code>edges</code>	array	Edge graph

Field	Type	Notes
agent_card	object null	Resolved/custom agent card payload
is_custom_agent_card	bool	
recaptcha_required	bool	Gates chat message add
accountId, accountName	string	Fixed "karexpert" / "Karexpert" today
g_creation_time, g_modify_time	int (ms)	
g_created_by_id, g_modified_by_id	string (UUID) null	
g_soft_delete	"Y" "N"	

Indexes: unique flow_name (partial: g_soft_delete="N").

users

Repository: `MongoUserRepository`. Application accounts.

Field	Type	Notes
_id	string (UUID)	
email	string	Unique
first_name, last_name	string	
password_hash	string	Bcrypt hash
g_creation_time, g_modify_time	int (ms)	No soft-delete field on this collection

Indexes: unique email.

embed_clients

Repository: `MongoEmbedClientRepository`. Embed-widget client credentials scoping which flows an embedded client may invoke.

Field	Type	Notes
_id	string (UUID)	
client_account_id	string	Unique among non-deleted
secret_hash	string	Hashed client secret
flow_ids	array<string UUID>	Flows this client can access

Field	Type	Notes
origins	array<string>	Allowed CORS origins for embed
enabled	bool	
g_creation_time, g_modify_time, g_created_by_id, g_modified_by_id, g_soft_delete		

Indexes: unique `client_account_id` (partial: `g_soft_delete="N"` and field exists).

projects

Repository: `MongoProjectRepository`. Multi-tenant project container that groups flows and exposes an MCP surface.

Field	Type	Notes
_id	string (UUID)	
name	string	Unique among non-deleted
description	string	
api_key	string	Unique (sparse)
g_creation_time, g_modify_time, g_created_by_id, g_modified_by_id, g_soft_delete		

Indexes: unique `name` (partial: `g_soft_delete="N"`); unique sparse `api_key`.

project_flows

Repository: `MongoProjectFlowRepository`. Join collection mapping a project's flows to MCP tool names. `_id` is the composite string "`<project_id>:<flow_id>`", written via `replace_one(..., upsert=True)`.

Field	Type	Notes
_id	string	"{project_id}:{flow_id}"
project_id, flow_id	string (UUID)	
tool_name	string	Unique per project; exposed as the MCP tool name
description	string	Tool description surfaced to MCP clients

Indexes: unique (`project_id`, `flow_id`); unique (`project_id`, `tool_name`).

mcp_execution_logs

Repository: `MongoMcpExecutionLogRepository`. Audit log of MCP tool invocations against a project.

Field	Type	Notes
<code>_id</code>	string (UUID)	
<code>project_id</code>	string (UUID)	
<code>flow_id</code>	string (UUID) null	
<code>tool_name</code>	string	
<code>args</code>	object null	Tool call arguments
<code>output</code>	any null	Tool result
<code>error</code>	string null	
<code>success</code>	bool	
<code>latency_ms</code>	int null	
<code>source</code>	string	Caller/source tag
<code>g_creation_time</code>	int (ms)	

Indexes: `project_id`. Listed newest-first via `sort("g_creation_time", -1)`.

chat_sessions

Repository: `MongoChatSessionRepository`. A conversation thread bound to one flow.

Field	Type	Notes
<code>_id</code>	string (UUID)	
<code>name</code>	string	
<code>flow_id</code>	string (UUID)	
<code>client_id</code>	string null	Embed client, if opened via embed
<code>client_user_id</code>	string null	End-user identity within the embed client
<code>g_creation_time, g_modify_time, g_created_by_id, g_modified_by_id, g_soft_delete</code>		

Indexes: `flow_id`; `client_user_id`.

chat_messages

Repository: `MongoChatMessageRepository`. Individual turns within a chat session.

Field	Type	Notes
<code>_id</code>	string (UUID)	
<code>session_id</code>	string (UUID)	
<code>text</code>	string	
<code>type</code>	string	e.g. user/assistant message type
<code>g_creation_time, g_modify_time, g_created_by_id, g_modified_by_id, g_soft_delete</code>		

Indexes: `session_id`.

providers

Repository: `MongoProviderRepository`. AI provider registry (OpenAI, Anthropic, Google, ...), upserted by name.

Field	Type	Notes
<code>_id</code>	string (UUID)	
<code>name</code>	string	Unique among non-deleted; upsert key
<code>display_name, icon, vendor_url</code>	string	
<code>auth_required</code>	bool	
<code>region</code>	string null	
<code>extra_metadata</code>	object	
<code>g_creation_time, g_modify_time, g_created_by_id, g_modified_by_id, g_soft_delete</code>		

Indexes: unique `name` (partial: `g_soft_delete="N"`).

provider_models

Repository: `MongoProviderModelRepository`. Models offered by a provider, upserted by (`provider_id`, `name`, `version`).

Field	Type	Notes
<code>_id</code>	string (UUID)	

Field	Type	Notes
<code>provider_id</code>	string (UUID)	FK → <code>providers._id</code>
<code>name, version, model_type, icon</code>	string	
<code>tool_calling, reasoning, search, preview, not_supported, deprecated, is_default</code>	bool	Capability/status flags
<code>max_input_tokens, max_output_tokens</code>	int null	
<code>latency_ms_p50</code>	number null	
<code>cost_per_1k_tokens, cost_per_1k_input_tokens, cost_per_1k_output_tokens</code>	number null	
<code>tags</code>	any	
<code>g_creation_time, g_modify_time, g_created_by_id, g_modified_by_id, g_soft_delete</code>		

Indexes: unique (`provider_id`, `name`, `version`).

`model_capabilities`

Repository: `MongoModelCapabilityRepository`. Capability rows per model, upserted by (`model_id`, `capability`).

Field	Type	Notes
<code>_id</code>	string (UUID)	
<code>model_id</code>	string (UUID)	FK → <code>provider_models._id</code>
<code>capability</code>	string	
<code>constraints</code>	object	
<code>extra_metadata</code>	object	
<code>g_creation_time, g_modify_time, g_created_by_id, g_modified_by_id, g_soft_delete</code>		

Indexes: unique (`model_id`, `capability`).

`agent_sessions`

Repository: `MongoAgentSessionRepository`. A configured provider/model/API-key session used to run a flow's agent nodes.

Field	Type	Notes
<code>_id</code>	string (UUID)	
<code>user_id</code>	string (UUID)	
<code>flow_id</code>	string (UUID)	
<code>provider_name, model_name</code>	string	
<code>api_key</code>	string	
<code>extra</code>	object	
<code>g_creation_time, g_modify_time, g_created_by_id, g_modified_by_id, g_soft_delete</code>		

Indexes: `flow_id`; `user_id`.

agent_memory

Repository: `MongoAgentMemoryRepository`. Conversational/long-term memory entries an agent node can recall by key. No soft-delete field — rows are hard-deleted.

Field	Type	Notes
<code>_id</code>	string (UUID)	
<code>user_id</code>	string (UUID)	
<code>session_id</code>	string null	
<code>flow_id</code>	string (UUID) null	
<code>memory_key</code>	string	Logical bucket for the memory (e.g. per-tool/topic)
<code>role</code>	string	e.g. user/assistant
<code>content</code>	any	
<code>g_creation_time</code>	int (ms)	

Indexes: `(user_id, memory_key)`; `(user_id, flow_id, memory_key)`; `flow_id`. Reads sort by `g_creation_time` desc and `limit`; deletes support by `(user_id, memory_key)`, `(user_id, session_id)`, `(user_id, flow_id)`.

usage_logs

Repository: `MongoUsageLogRepository`. Per-LLM-call usage/cost/latency accounting.

Field	Type	Notes
<code>_id</code>	string (UUID)	
<code>session_id, flow_id, provider_id, model_id</code>	string (UUID)	
<code>provider_name, model_name</code>	string	
<code>capability</code>	string	
<code>tokens_used</code>	int	
<code>latency_ms</code>	int	
<code>cost</code>	number	
<code>timestamp</code>	int (ms)	

Indexes: `flow_id`.

`tool_call_logs`

Repository: `MongoToolCallLogRepository`. Individual tool-call records nested under a usage log / agent run.

Field	Type	Notes
<code>_id</code>	string (UUID)	
<code>usage_log_id, session_id, flow_id</code>	string (UUID)	
<code>node_id, tool_call_id, tool_name</code>	string	
<code>tool_args, tool_result</code>	any	
<code>g_creation_time</code>	int (ms)	

Indexes: `flow_id`.

`flow_runs`

Repository: `MongoFlowRunRepository`. Async flow-execution job queue backing `POST /flows/{flow_id}/runs`, with retry/idempotency support.

Field	Type	Notes
<code>_id</code>	string (UUID)	
<code>flow_id</code>	string (UUID)	
<code>status</code>	string	<code>PENDING</code> <code>RUNNING</code> terminal states (<code>SUCCEEDED</code> / <code>FAILED</code> /etc.)

Field	Type	Notes
<code>requested_by</code>	string (UUID) null	
<code>g_creation_time, g_modify_time</code>	int (ms)	
<code>started_at, finished_at</code>	int (ms) null	
<code>idempotency_key</code>	string null	Dedupe key per <code>flow_id</code>
<code>input_payload, output_payload</code>	any	
<code>error</code>	string null	
<code>max_retries, retry_backoff_sec, max_concurrency</code>	int	
<code>cancellation_requested</code>	bool	

Indexes: `flow_id`; `idempotency_key`; unique (`flow_id`, `idempotency_key`) (partial: `idempotency_key` is a string); (`status`, `g_creation_time`) — powers `claim_next_pending()`, which atomically claims the oldest **PENDING** run (or a stale **RUNNING** run past `stale_before_ms`) via `find_one_and_update` and flips it to **RUNNING**.

flow_logs

Repository: `MongoFlowLogRepository`. Per-node execution log entries written during a flow run (mirrors the **FLOW_LOGS** SQL table).

Field	Type	Notes
<code>_id</code>	string (UUID)	
<code>flow_id</code>	string (UUID)	
<code>node_id, node_type</code>	string	
<code>timestamp</code>	int (ms)	
<code>inputs, outputs</code>	string (serialized)	
<code>status</code>	string	
<code>error</code>	string null	

Indexes: `flow_id`.

Collection index summary

Collection	Index	Type
flows	flow_name	unique, partial (g_soft_delete="N")
users	email	unique
embed_clients	client_account_id	unique, partial
flow_runs	flow_id	—
flow_runs	idempotency_key	—
flow_runs	(flow_id, idempotency_key)	unique, partial (string key)
flow_runs	(status, g_creation_time)	—
flow_logs	flow_id	—
projects	name	unique, partial
projects	api_key	unique, sparse
project_flows	(project_id, flow_id)	unique
project_flows	(project_id, tool_name)	unique
mcp_execution_logs	project_id	—
chat_sessions	flow_id	—
chat_sessions	client_user_id	—
chat_messages	session_id	—
providers	name	unique, partial
provider_models	(provider_id, name, version)	unique
model_capabilities	(model_id, capability)	unique
agent_sessions	flow_id	—
agent_sessions	user_id	—
agent_memory	(user_id, memory_key)	—
agent_memory	(user_id, flow_id, memory_key)	—
agent_memory	flow_id	—
usage_logs	flow_id	—
tool_call_logs	flow_id	—

Indexes are created (idempotently, warnings-only on conflict) by `ensure_mongo_indexes()` at startup when `DB_BACKEND=mongo`.

API Architecture

API Route Structure

```

/auth
  POST / - Register new user (rate limited: 5/hour)
  POST /token - Login and get JWT token
  POST /embed-token - Issue embed-scoped token

/users
  GET /me - Get current user profile
  PUT /change-password - Change password

/agents
  GET /registry - List flow-derived agent registry

/flows
  POST / - Create new flow
  GET /all - List all flows (non-paginated)
  GET / - List flows (paginated)
  GET /{flow_id} - Get flow by ID
  PUT /{flow_id} - Update flow
  DELETE /{flow_id} - Soft delete flow
  POST /{flow_id}/execute - Execute flow immediately
  POST /{flow_id}/runs - Create async flow run
  GET /runs/{run_id} - Get async flow run status
  POST /runs/{run_id}/cancel - Cancel async flow run
  GET /{flow_id}/agent-card.json - Resolve agent card
  POST /{flow_id}/agent-card/save-resolved - Persist resolved agent
card

/chat
  POST /session - Create chat session
  GET /sessions - List sessions for flow (query:
flow_id)
  GET /messages/{session_id} - Get messages
  POST /message - Add message
  POST /tts - Generate speech audio

/projects
  POST / - Create project
  GET / - List projects (paginated)
  GET /{project_id} - Get project by ID
  PUT /{project_id} - Update project
  DELETE /{project_id} - Delete project
  GET /{project_id}/mcp-config - Get project MCP config
  GET /{project_id}/mcp-status - Get project MCP status
  GET /{project_id}/mcp-tools - List MCP tools
  POST /{project_id}/mcp-invoke - Invoke MCP tool
  GET /{project_id}/mcp-logs - List MCP execution logs
  POST /{project_id}/rotate-key - Rotate project API key

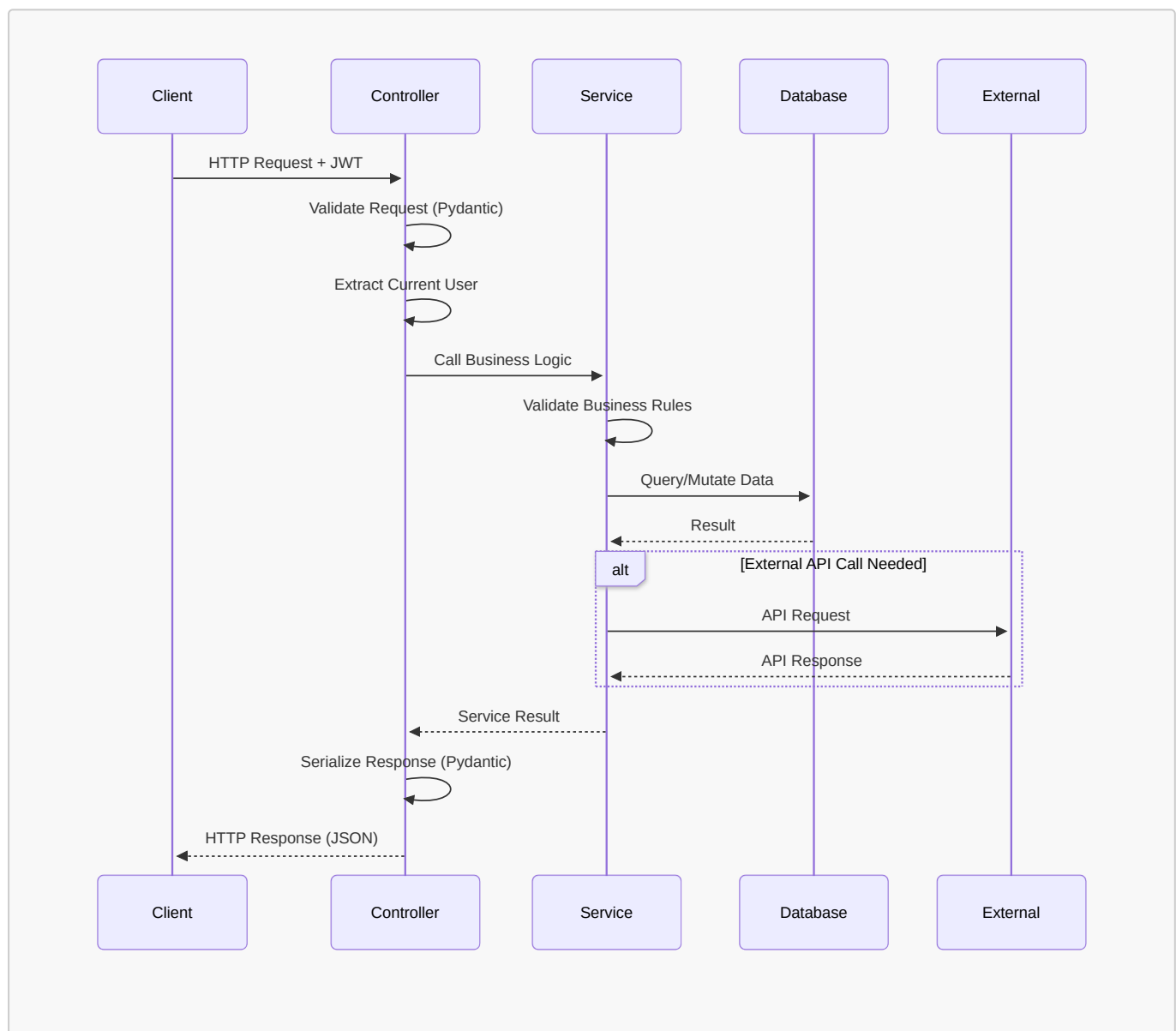
```

```
/embed-clients
  POST /          - Create embed client
  GET  /          - List embed clients (paginated)
  GET  /{client_id} - Get embed client
  PUT  /{client_id} - Update embed client
  DELETE /{client_id} - Soft delete embed client
  POST /{client_id}/rotate-secret - Rotate client secret

/agent_nodes
  GET  /providers - List AI providers
  GET  /models    - List provider models
  GET  /metadata  - List provider/model metadata
  GET  /git/branches - List git branches (GitHub/GitLab)

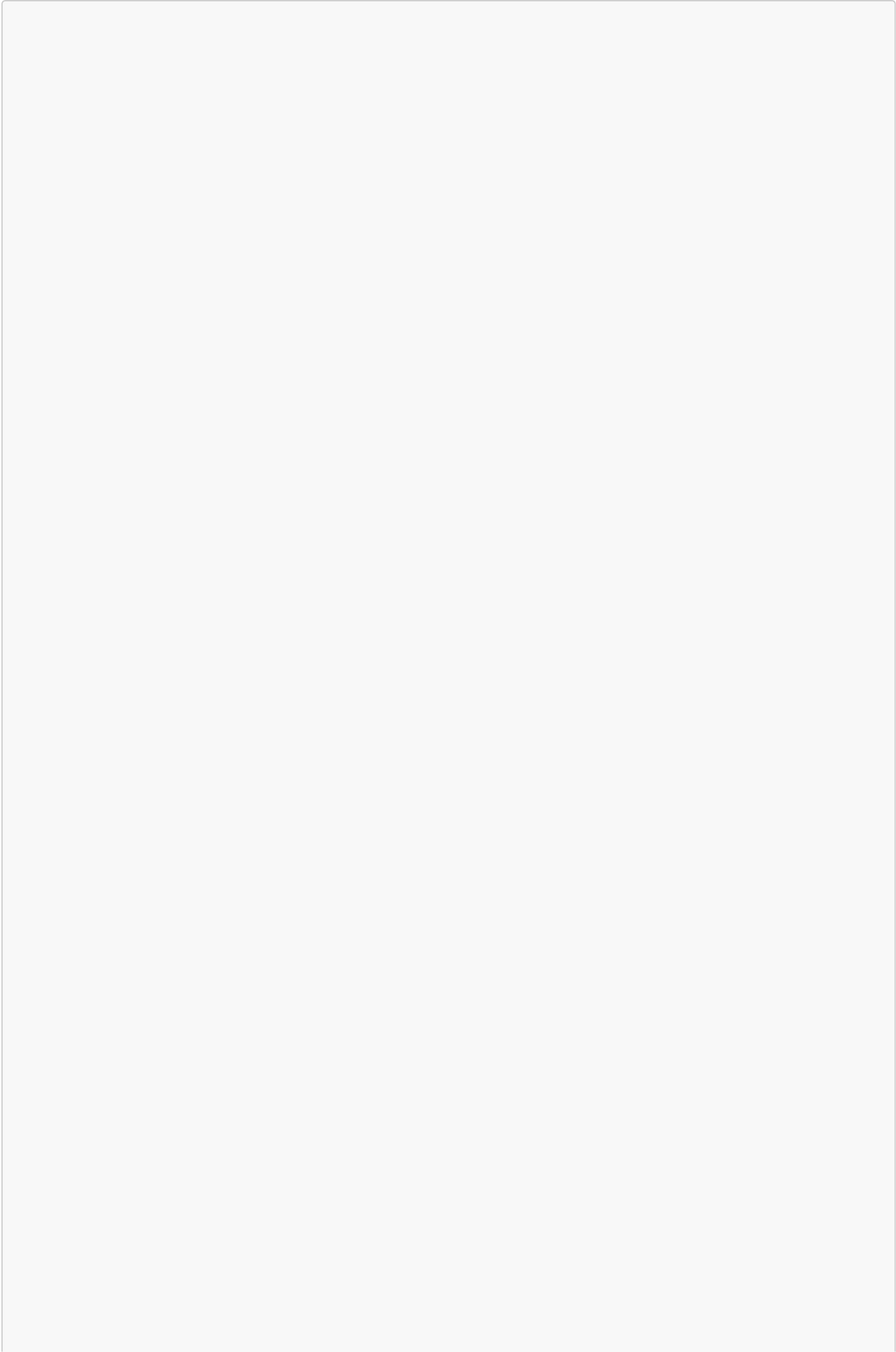
/whatsapp
  GET  /webhook    - Verify webhook
  POST /webhook    - Receive webhook events
```

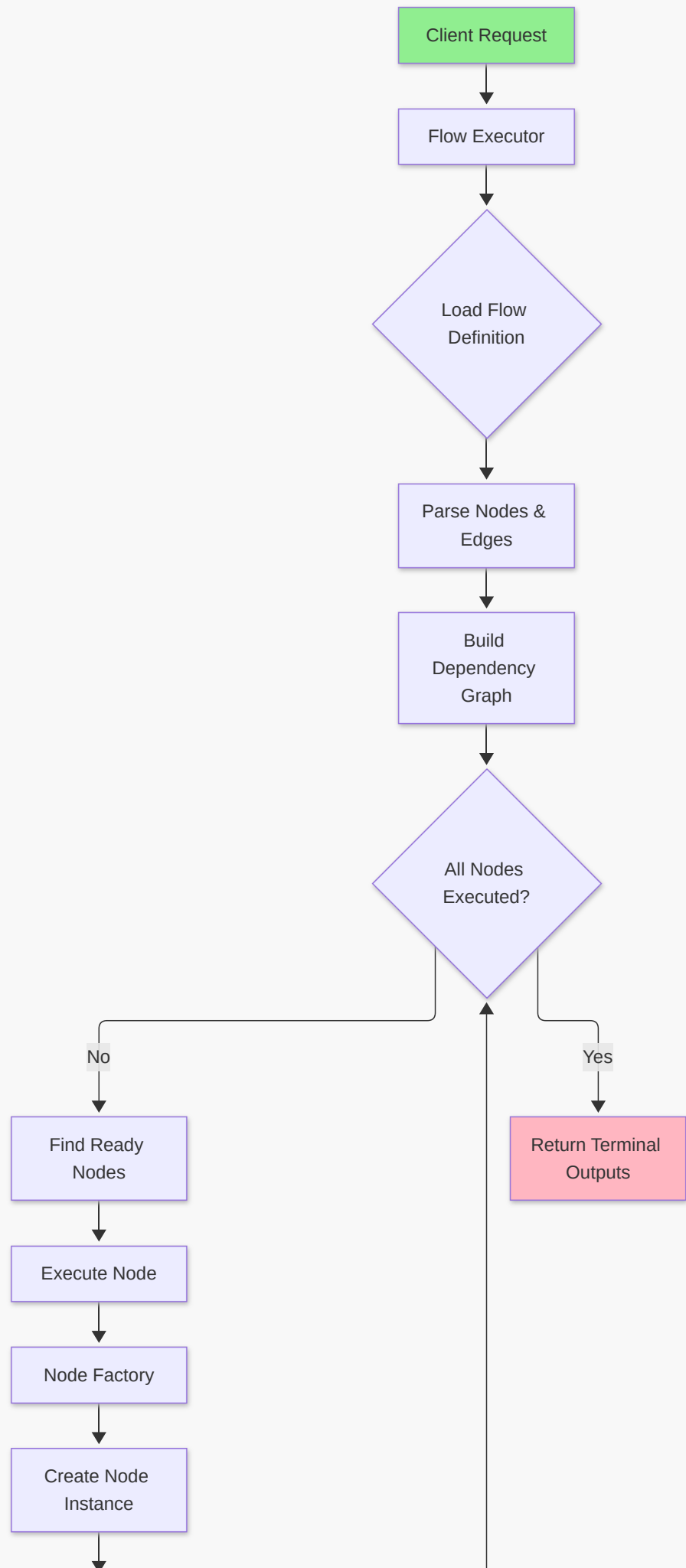
Request/Response Flow

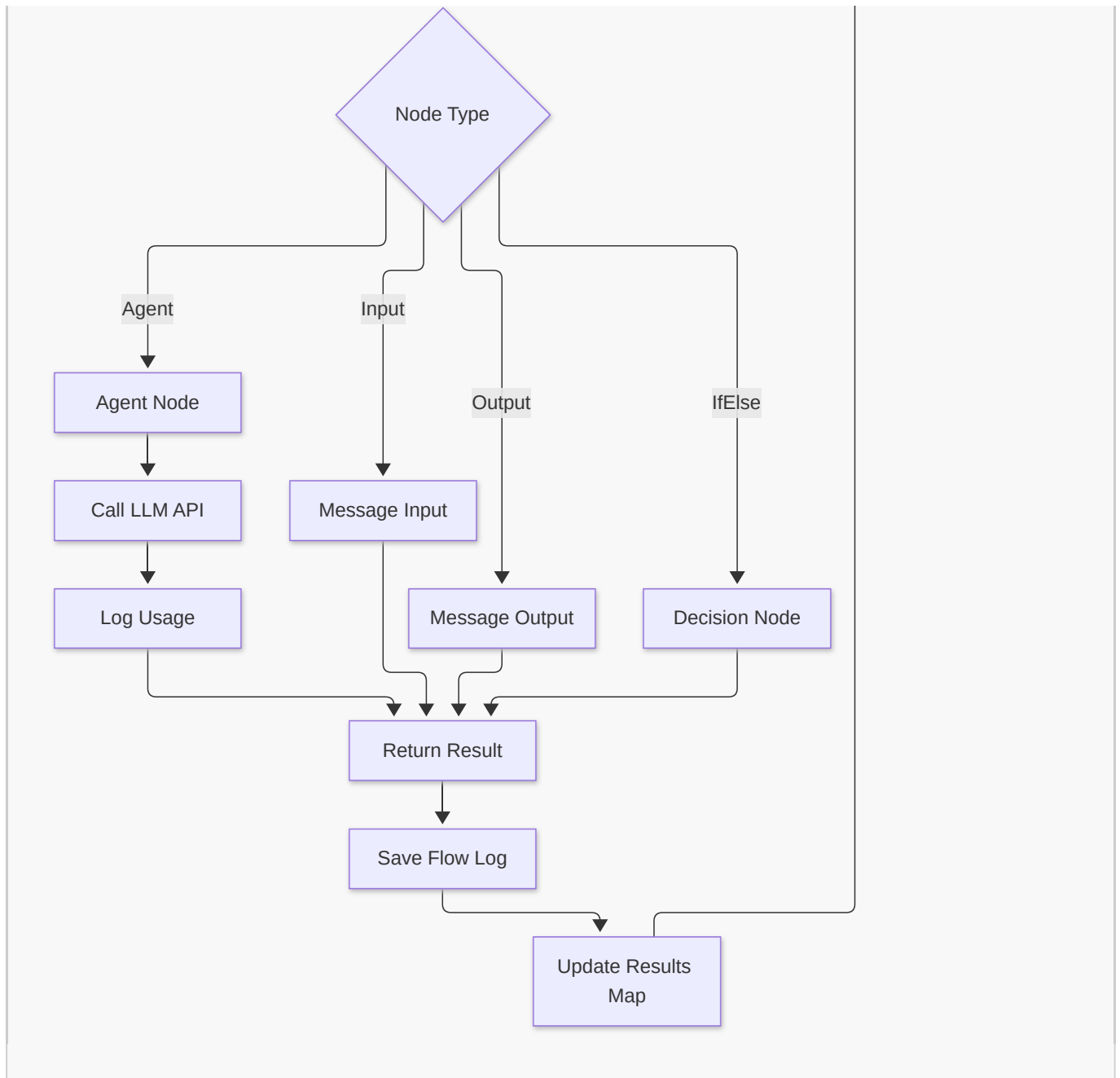


Workflow Engine

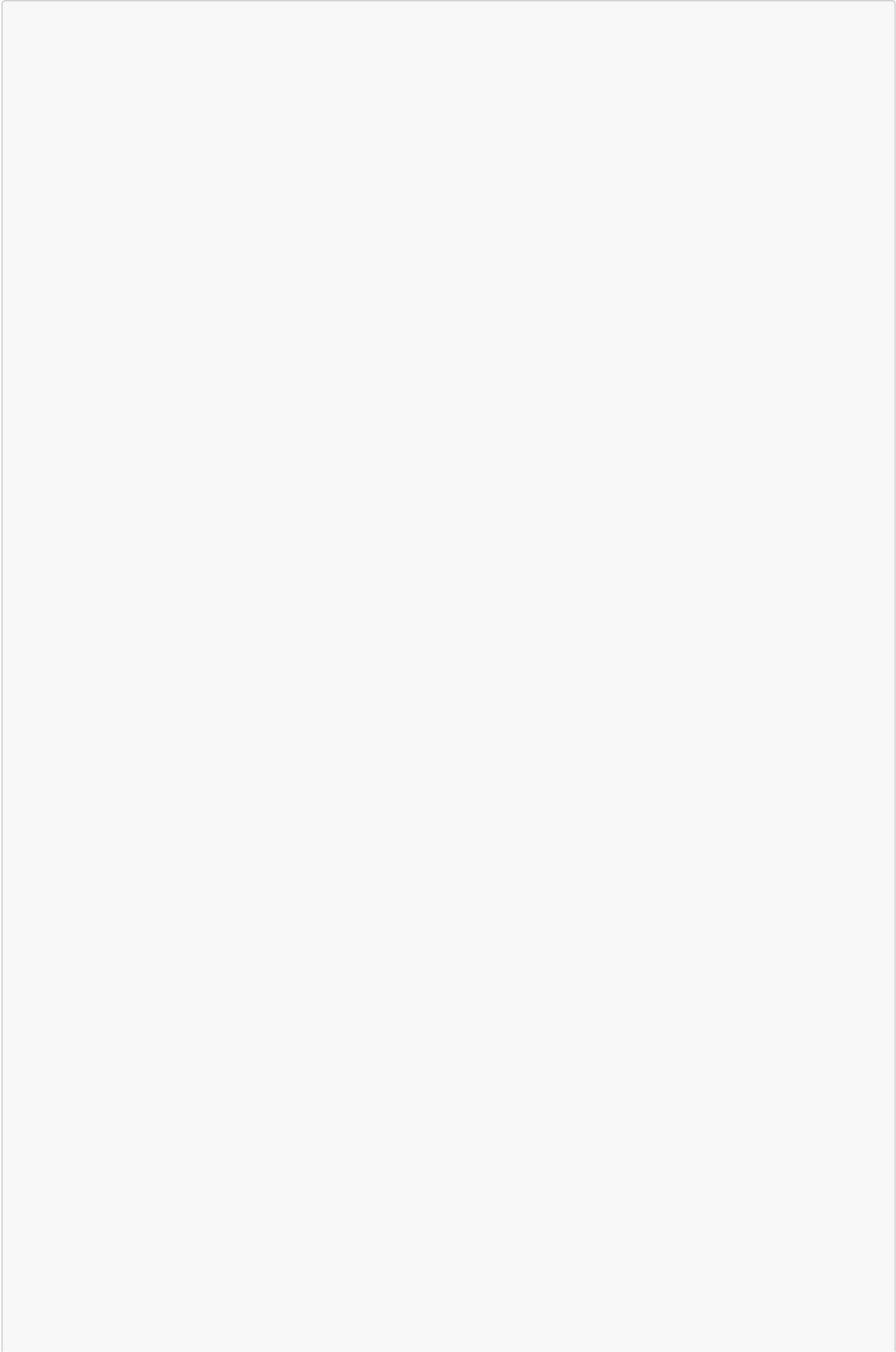
Workflow Execution Architecture

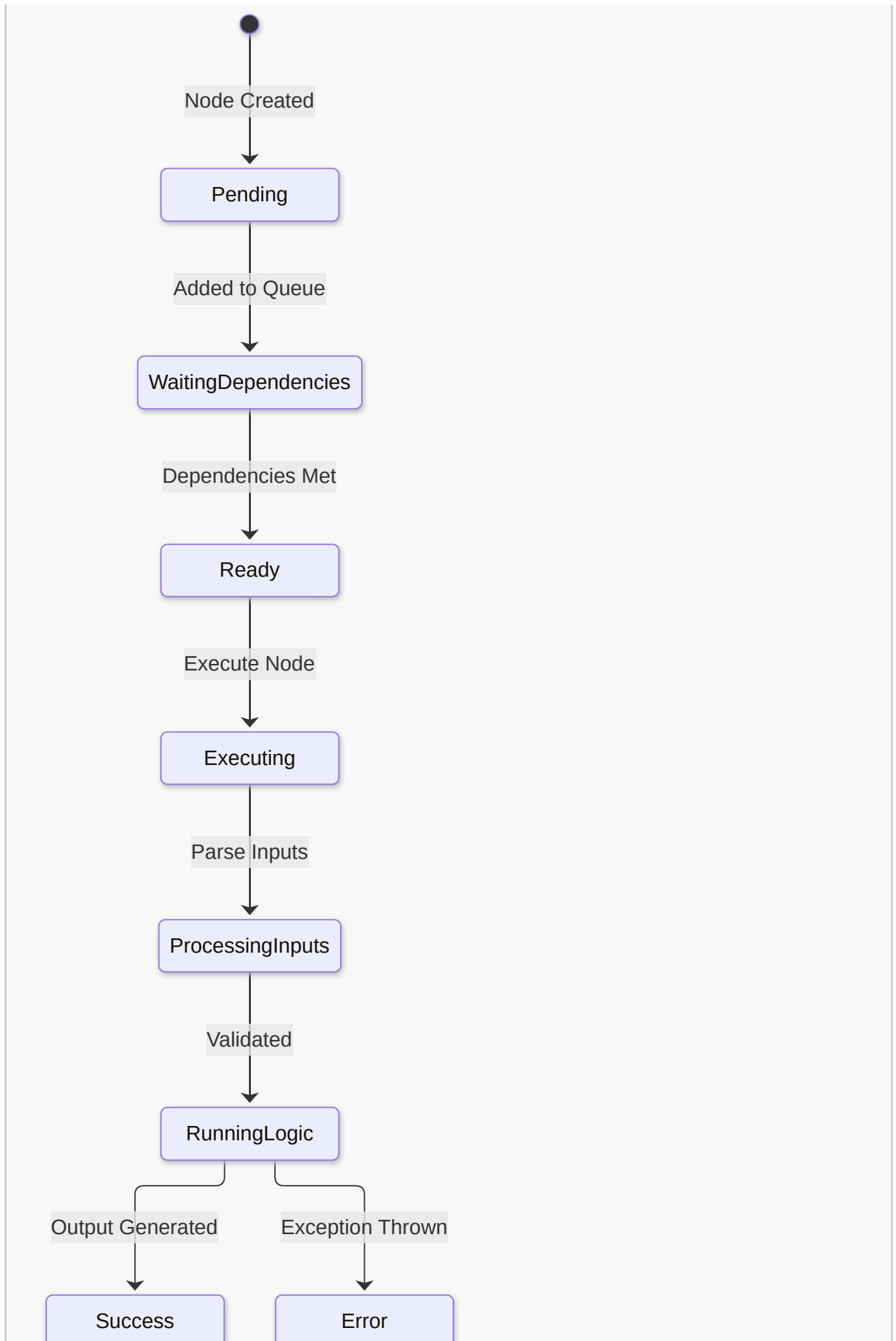


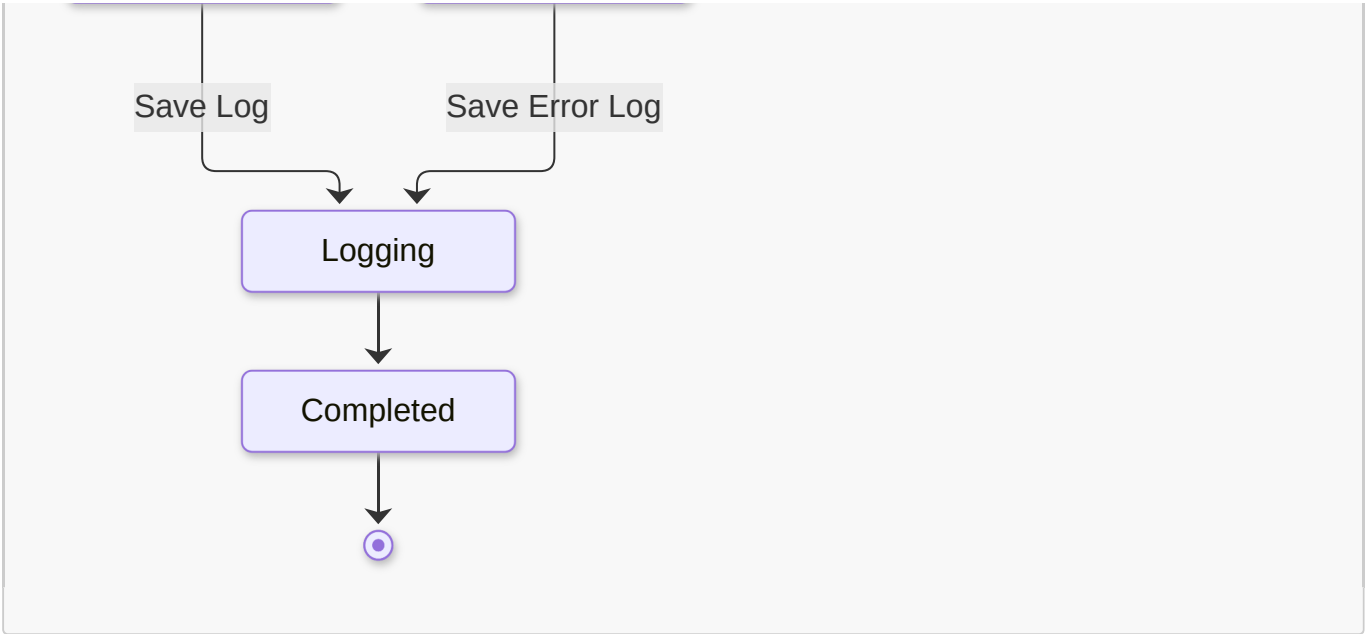




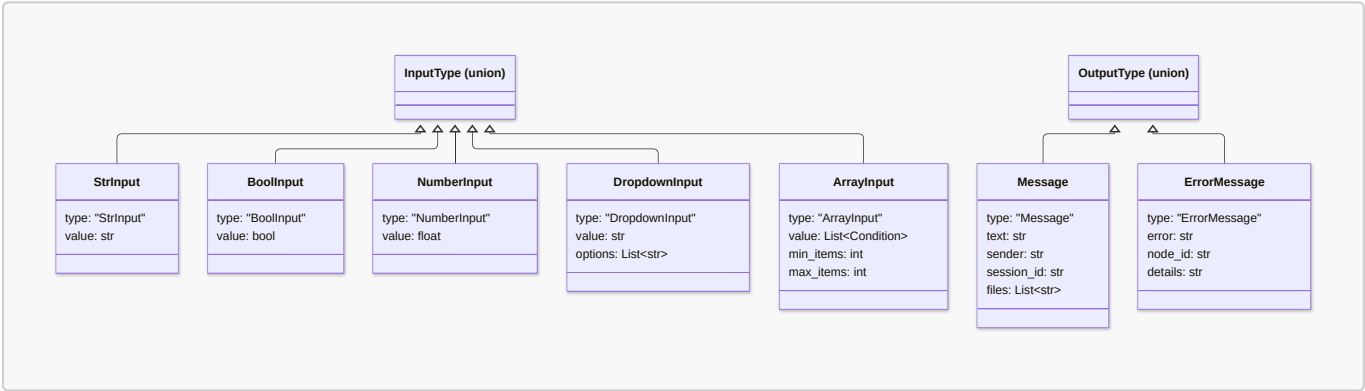
Node Execution Lifecycle





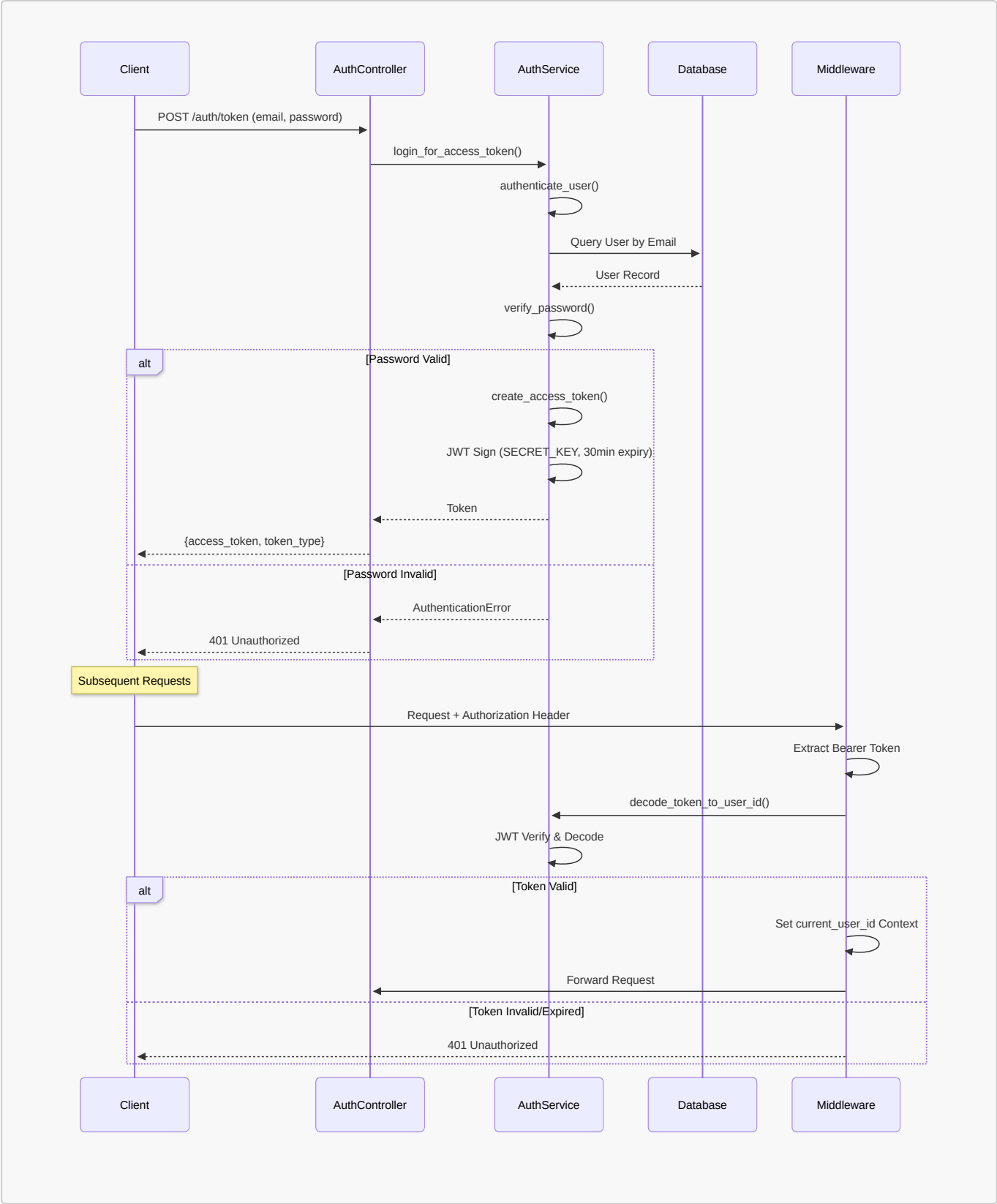


Input/Output Type System

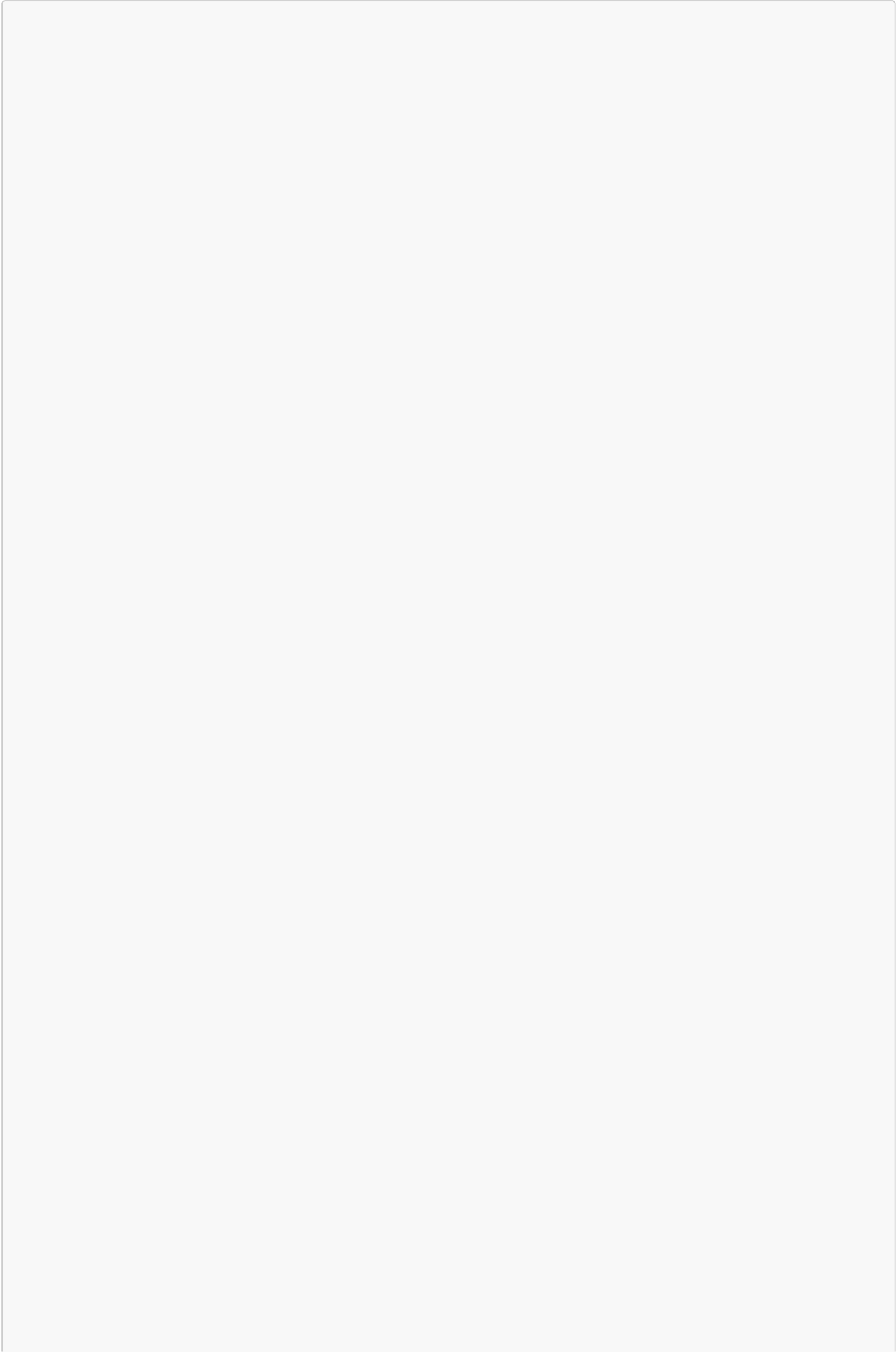


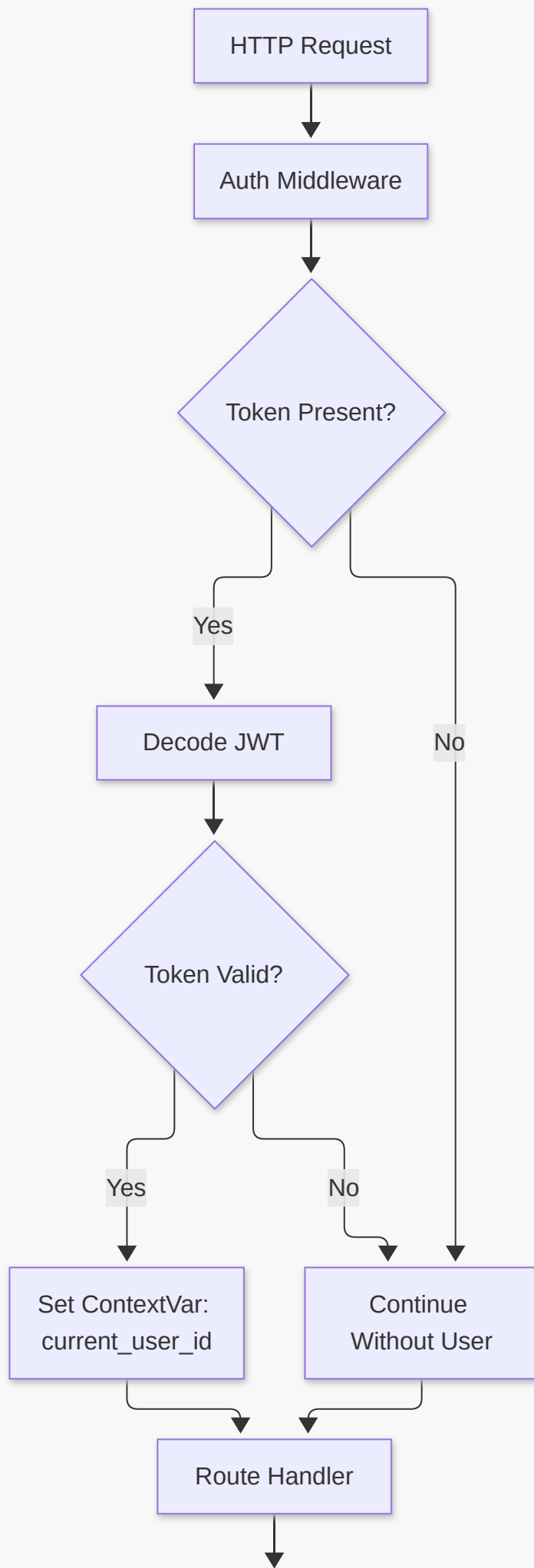
Authentication & Authorization

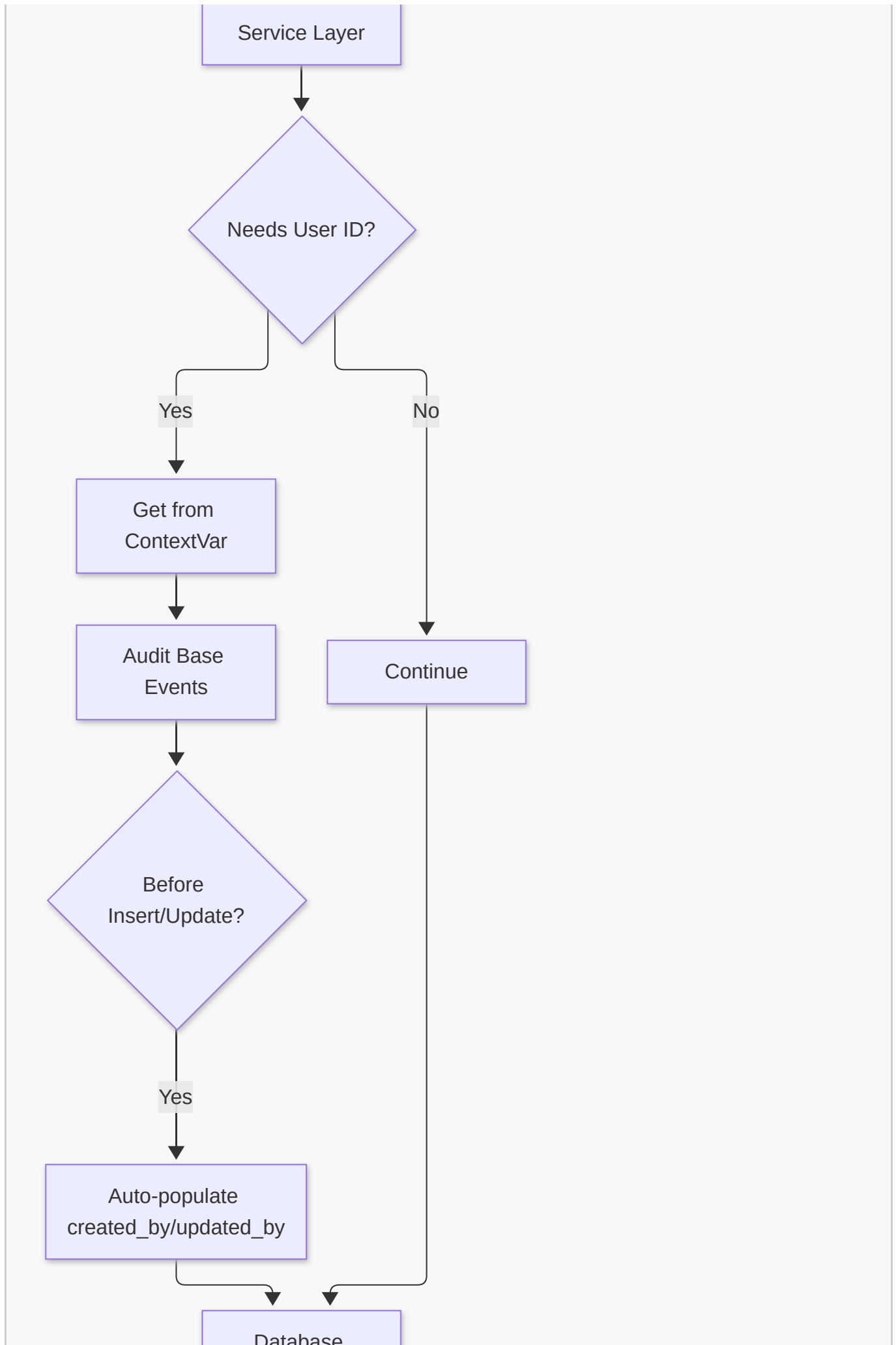
JWT Authentication Flow



User Context Management



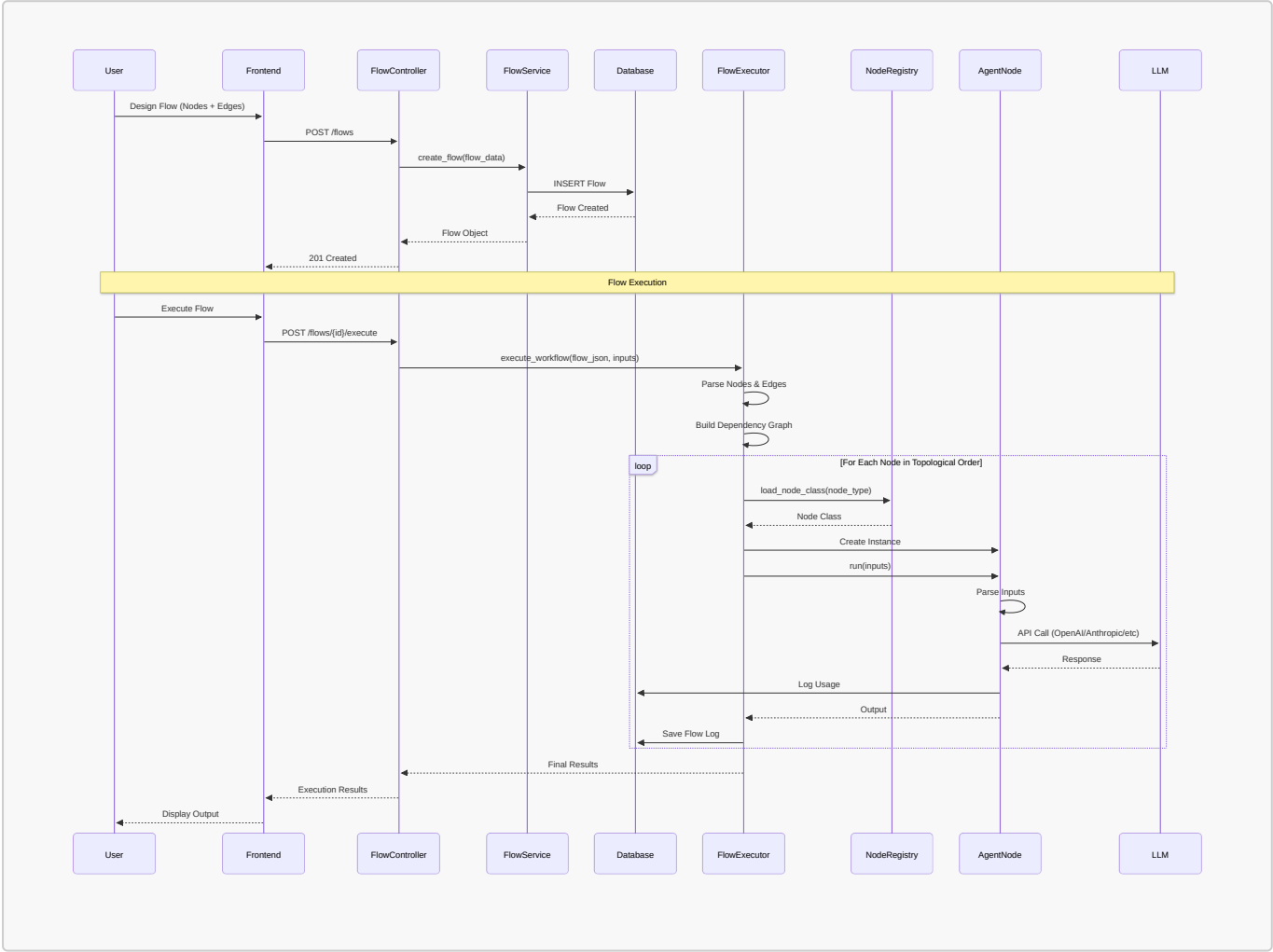




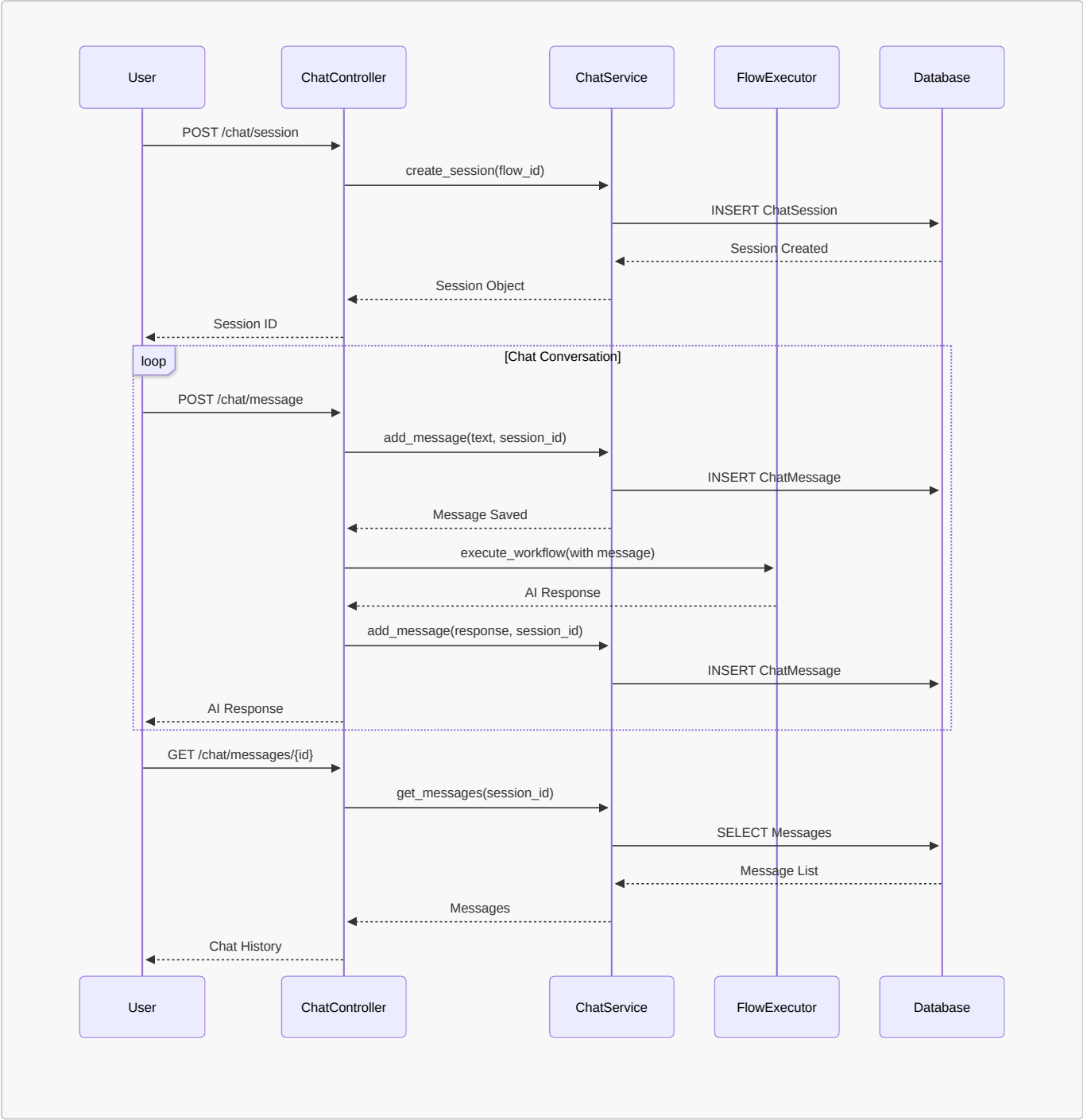
Operation

Sequence Diagrams

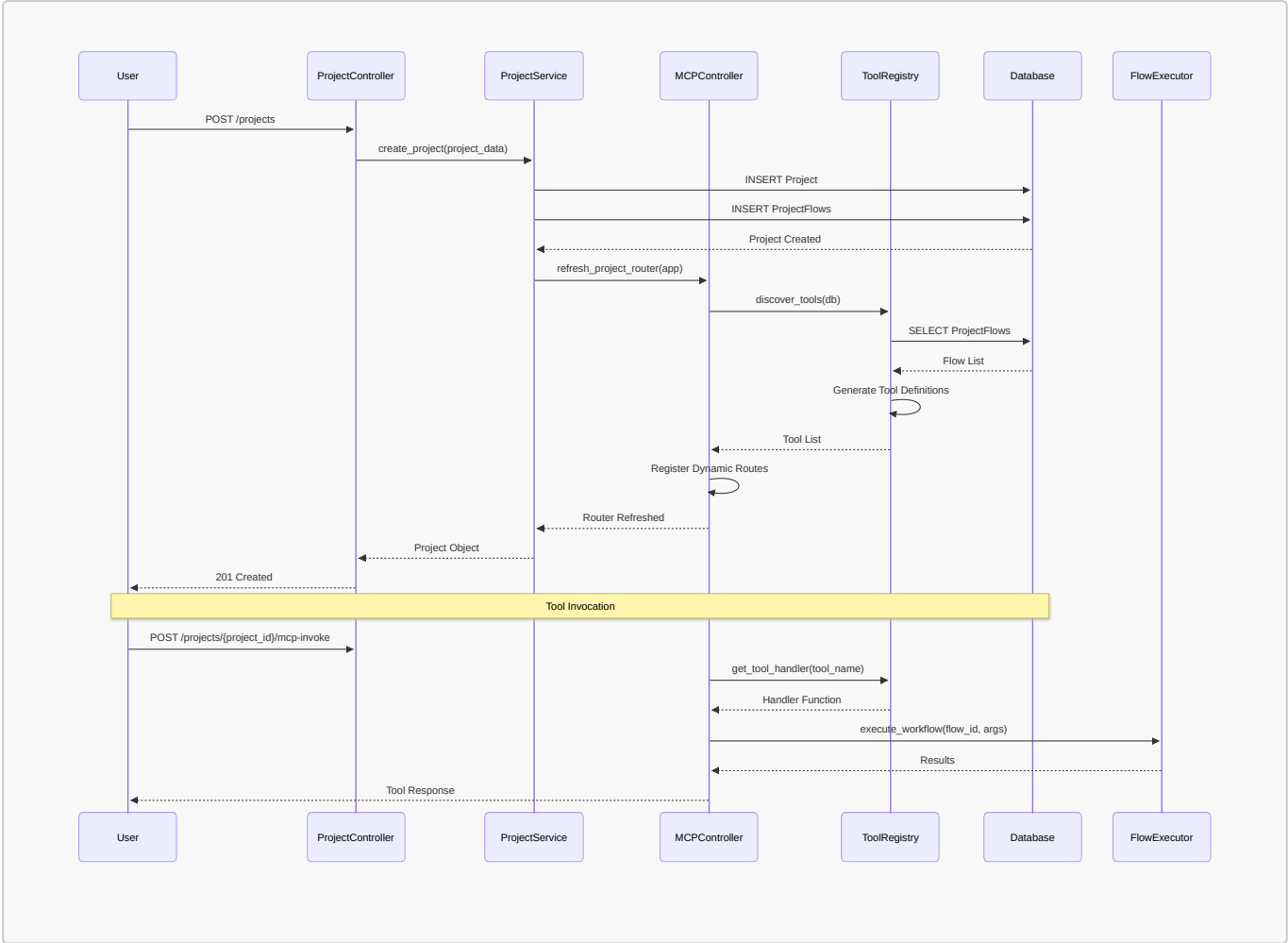
1. Flow Creation and Execution



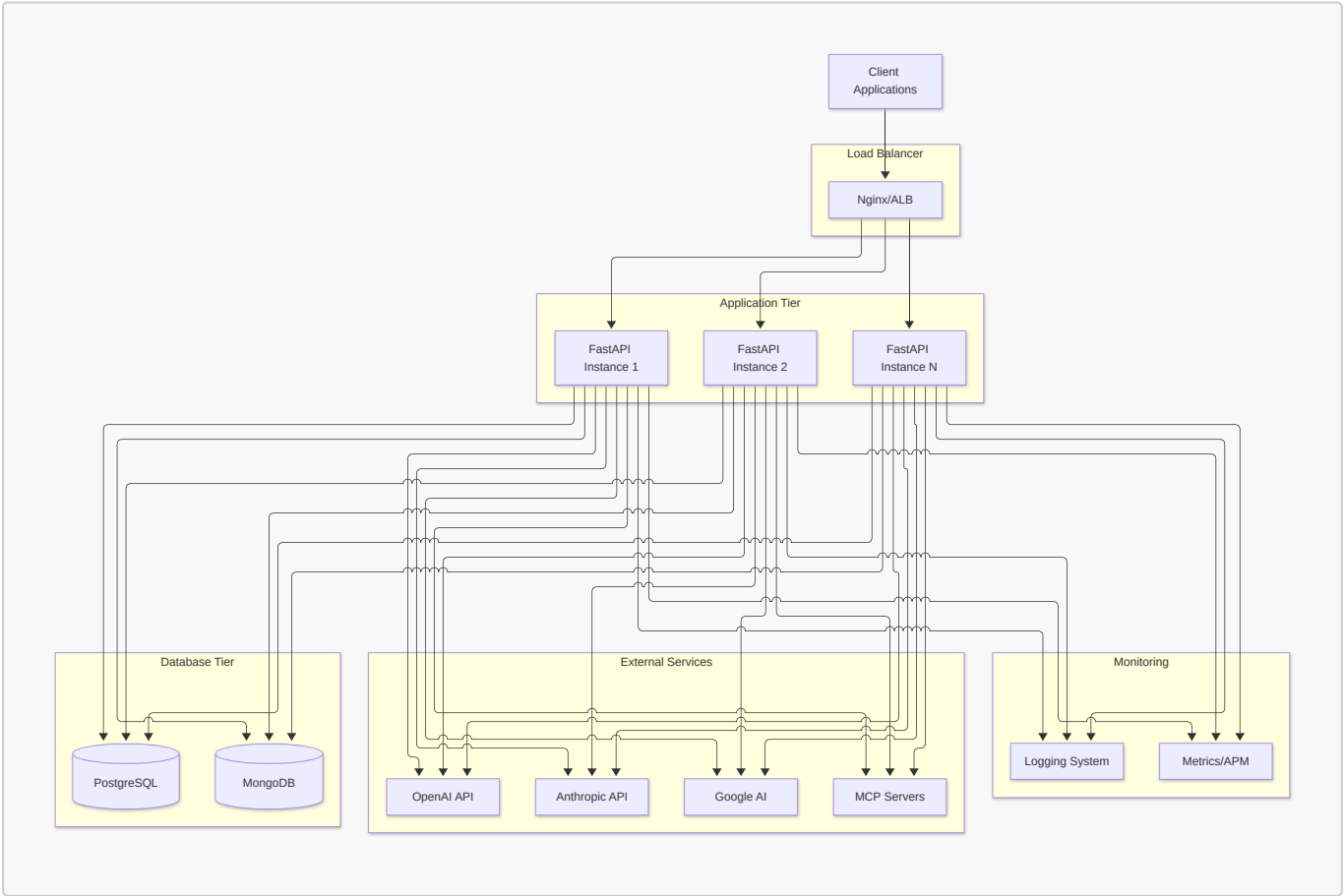
2. Chat Session with Flow



3. Project and MCP Tool Registration



Deployment Architecture



Deployment Components

1. **Load Balancer:** Distributes traffic across API instances
 2. **Application Tier:** Horizontally scalable FastAPI instances
 3. **Database Tier:** Pluggable persistence backend (PostgreSQL or MongoDB)
 4. **External Services:** AI providers and MCP servers
 5. **Monitoring:** Centralized logging and metrics
-

Key Technical Decisions

1. Why FastAPI?

- **Async Support:** Native async/await for concurrent operations
- **Performance:** One of the fastest Python frameworks
- **Type Safety:** Pydantic integration for request/response validation
- **Auto Documentation:** OpenAPI/Swagger generation
- **Modern:** Type hints, dependency injection

2. Why SQLAlchemy?

- **ORM Benefits:** Pythonic database operations
- **Database Agnostic:** Easy to switch databases
- **Migration Support:** Alembic integration
- **Relationship Management:** Foreign keys, joins
- **Async Support:** Async engine and sessions

3. Why Pluggable Database Backend (PostgreSQL + MongoDB)?

- **Operational Flexibility:** Select backend via `DB_BACKEND` without changing route/service contracts
- **Shared Domain Model:** Repository layer keeps business logic stable across SQL and document storage
- **Backend-Specific Optimization:** SQLAlchemy for relational workflows, Mongo indexes for high-volume event/chat data
- **Migration Path:** Scripts and repositories support gradual movement between backends
- **Environment Fit:** Teams can optimize for transactional or document-centric workloads

4. Why Context Variables?

- **Thread Safety:** Async-safe global state
- **Request Isolation:** Each request has own context
- **Clean Code:** No need to pass `user_id` everywhere
- **Middleware Integration:** Set once, use everywhere

5. Why Soft Deletes?

- **Data Recovery:** Can restore deleted items
 - **Audit Trail:** Maintain history
 - **Referential Integrity:** No orphaned relationships
 - **Compliance:** Meet data retention requirements
-

Security Considerations

1. Authentication

- JWT access tokens with configurable expiry (`ACCESS_TOKEN_EXPIRE_MINUTES`, default 60)
- Dedicated embed tokens signed with `EMBED_SECRET_KEY`
- Bcrypt password hashing
- Bearer token authentication
- Secret key stored in environment

2. Authorization

- User context in ContextVar
- Automatic audit trail (`created_by`, `updated_by`)
- Rate limiting on sensitive endpoints
- CORS configuration

3. Input Validation

- Pydantic models validate all inputs
- Type checking prevents injection
- SQL injection prevented by ORM
- API key validation for projects

4. Data Protection

- Passwords hashed, never stored plain
- API keys for external services
- Soft deletes maintain data
- Audit logs for compliance

Performance Optimizations

1. Database Level

- Indexed columns (`id`, `email`, `flow_name`)
- Connection pooling via `SessionLocal`
- Mongo collection indexes for flow runs, chat, providers, projects, and embed clients
- Lazy loading relationships
- Query optimization with `select/join`

2. Application Level

- Async operations throughout
- Context variables for request scope
- Dependency injection for shared resources
- Rate limiting prevents abuse

3. Caching Strategies

- Provider/Model registry cached in memory
- Node class loading cached
- Tool handlers registered once

4. Scalability

- Stateless API design
- Horizontal scaling ready
- Database read replicas
- Load balancer distribution

Testing Strategy

1. Unit Tests

- Service layer tests (`test_auth_service.py`, `test_users_service.py`)
- Node execution tests
- Validation tests

2. Integration Tests

- E2E flow execution tests
- Database integration tests
- API endpoint tests

3. Test Infrastructure

- pytest framework
- `confptest.py` for fixtures
- Mock external APIs
- Test database isolation

Future Enhancements

1. **WebSocket Support:** Real-time flow execution updates
2. **Caching Layer:** Redis for session/token caching
3. **Batch Processing:** Queue system for long-running flows
4. **Monitoring:** Prometheus/Grafana integration
5. **API Versioning:** v1, v2 API namespaces
6. **Multi-tenancy:** Organization-level isolation
7. **Audit Logging:** Comprehensive audit trail
8. **Rate Limiting:** More granular per-user limits

Conclusion

The KX-AI Flow backend is built with modern Python best practices, leveraging FastAPI's performance and type safety. The architecture is clean, maintainable, and scalable, with clear separation of concerns across

layers. Design patterns are used appropriately to solve specific problems, and the codebase is structured for growth and extensibility across both SQL and document persistence backends.

Architecture Strengths

- ✓ Clean layered architecture
- ✓ Multiple design patterns properly applied
- ✓ Type-safe with Pydantic validation
- ✓ Async-first for performance
- ✓ Comprehensive audit trail
- ✓ Extensible workflow engine
- ✓ Multi-provider AI integration

Recommended Next Steps

1. Add comprehensive API documentation
2. Implement WebSocket for real-time updates
3. Add Redis caching layer
4. Implement API rate limiting per user
5. Add comprehensive monitoring and alerting
6. Implement database migrations with Alembic
7. Add integration test coverage

Document Maintained By: Sanjay Kumar Prajapati (AVP - Software Development)

Last Updated: September 4, 2026

Version: 1.2 (added MongoDB Collections reference)